Algorithms

Lecture 3
Sorting with Divide and Conquer

Muhammad Ehsanul Kader
Lecturer
Department of Computer Science and Engineering
BRAC University

Stable Sorting algorithm

A sorting algorithm is **stable** if it maintains the relative order of equal elements. For example, if two elements have the same value, their order remains the same as in the input array.

Example

Input: [(4, 'apple'), (2, 'banana'), (4, 'pear')]

Stable Sort Result (sorted by the left element): [(2, 'banana'), (4, 'apple'), (4, 'pear')]

Unstable Sort Result (sorted by the left element): [(2, 'banana'), (4, 'pear'), (4, 'apple')]

Stable sorting is important when multiple criteria are involved, and the order of equal elements matters.

Inplace Sorting algorithm

An **in-place** sorting algorithm uses a constant amount of extra space for producing the final sorted array. The sorting is done within the original array with only a small, constant amount of extra storage.

Example

Quick Sort: Uses a divide-and-conquer approach but sorts elements within the array.

Insertion Sort: Inserts elements into their correct position within the array without needing extra space.

Bubble Sort

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n-1):
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                temp = arr[j]
                arr[j] = arr[j+1]
                arr[j+1] = temp
```

Worst Case Complexity?
Best Case Complexity?
Space Complexity?
Optimization?

Bubble Sort: Complexity & Optimization

Idea: repeatedly compare adjacent elements and swap if they are in the wrong order.

One pass bubbles the largest value to the end



compare → swap

end of pass: last elements are sorted

Worst case	Best case	Space
Reverse sorted $O(n^2)$ comparisons + swaps	Already sorted $O(n^2)$ without early stop $O(n)$ with swapped flag	In-place sorting $O(1)$ extra space

Why $O(n^2)$?

$$(n-1) + (n-2) + \ldots + 1 = n(n-1)/2 \Rightarrow O(n^2)$$

Optimization: shrink inner loop because the last i elements are already fixed.

Optimized Bubble Sort

```
for i = 0 to n - 2:
  swapped = false

  for j = 0 to n - 2 - i:
    if a[j] > a[j+1]:
      swap(a[j], a[j+1])
      swapped = true

  if swapped == false:
    break
```

Key takeaway: without early stopping, best case is still $O(n^2)$; with the swapped flag, best case becomes $O(n)$.

Selection Sort

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

Worst Case Complexity?
Best Case Complexity?
Space Complexity?
Optimization?

Insertion Sort

```
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr [j+1] = arr[j]
            j = j -1
        arr[j+1] = key
```

Worst Case Complexity?
Best Case Complexity?
Space Complexity?
Optimization?

Divide and Conquer Strategy

Divide	the problem (instance) into subproblems.
Conquer	the subproblems by solving these recursively (or iteratively).
Combine	the solutions to the subproblems.

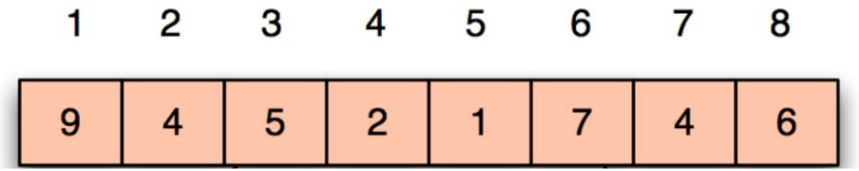
Merge Sort

The problem

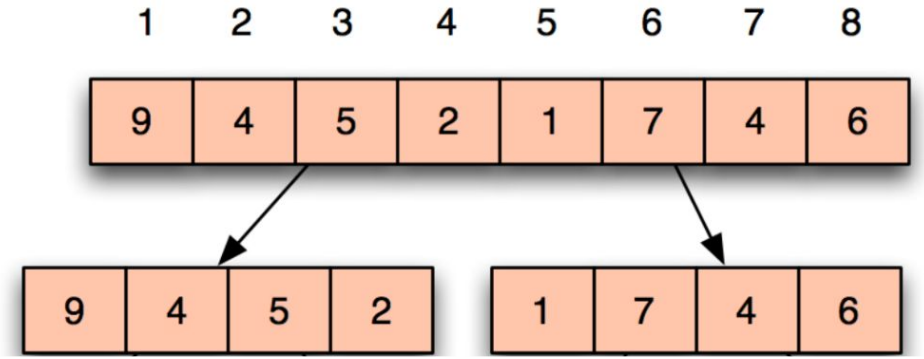
Sort an Array:

1. *Divide:* Trivial.
2. *Conquer:* Recursively sort 2 subarrays.
3. *Combine:* Merge the sorted subarrays in $\Theta(n)$ time.

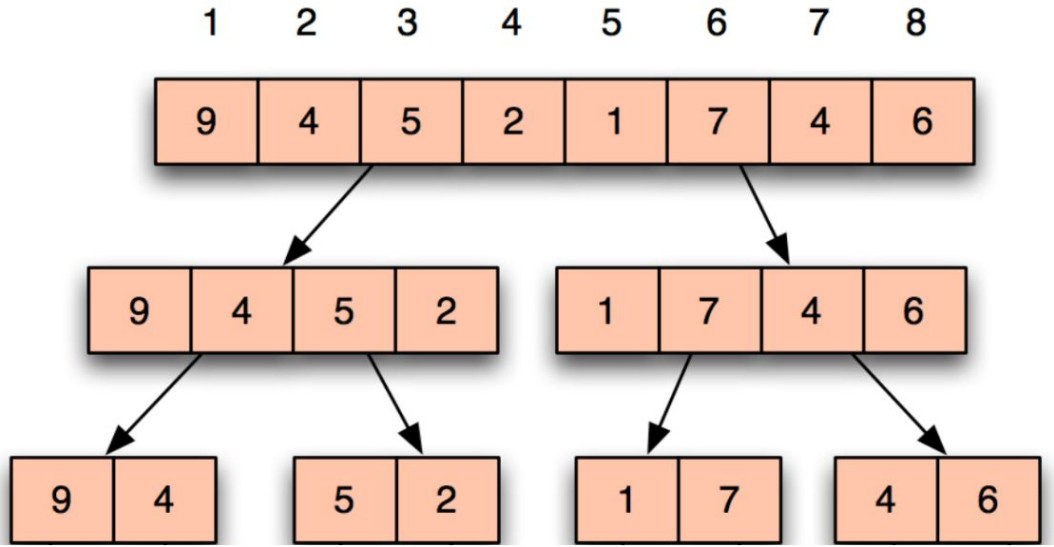
Merge Sort (Simulation)



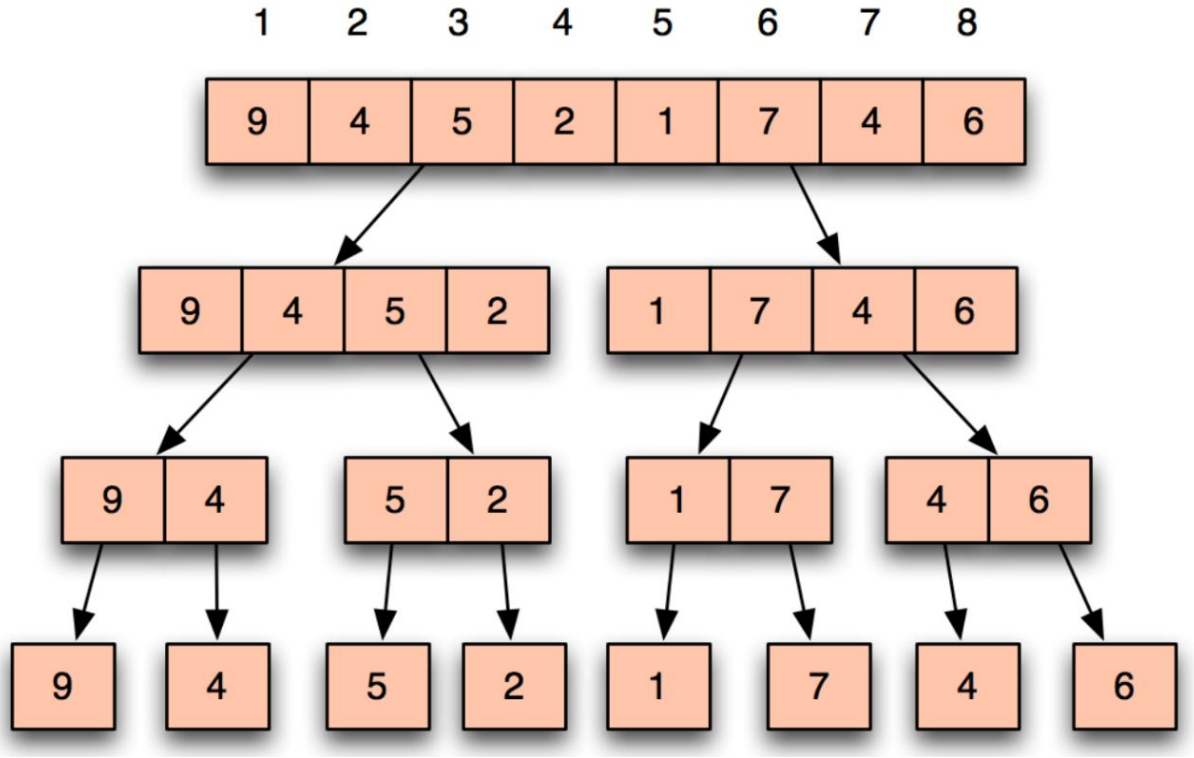
Merge Sort (Simulation)



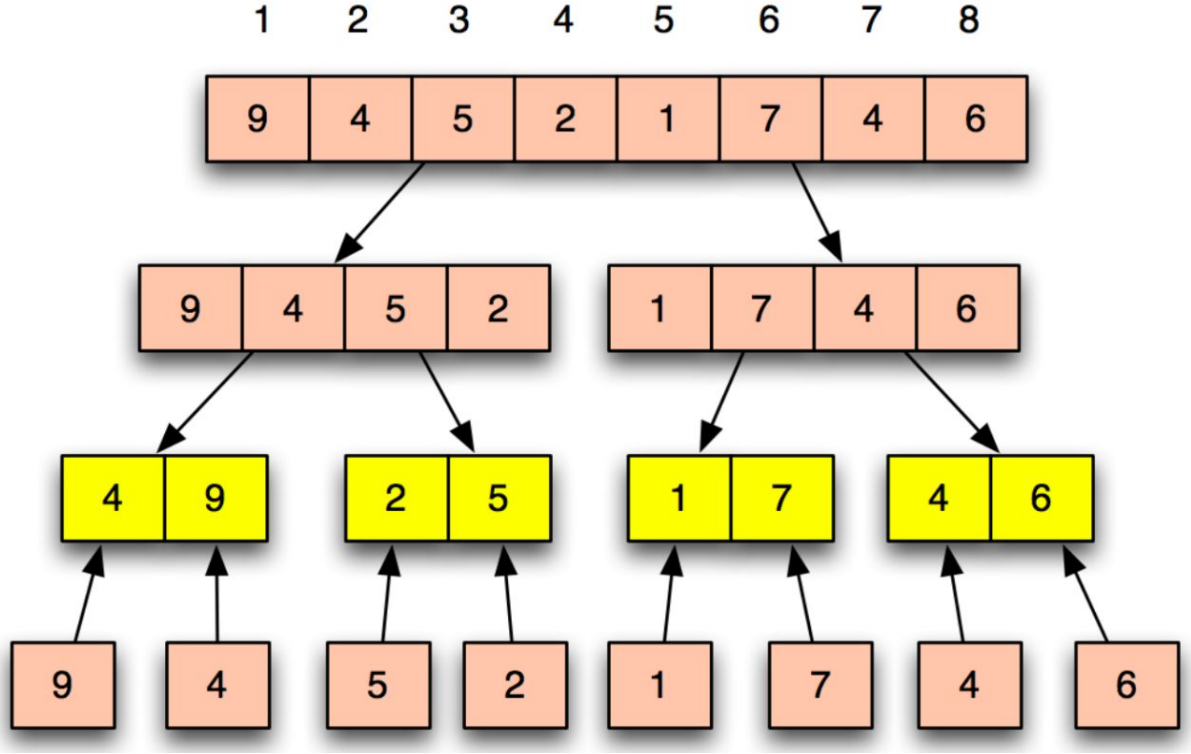
Merge Sort (Simulation)



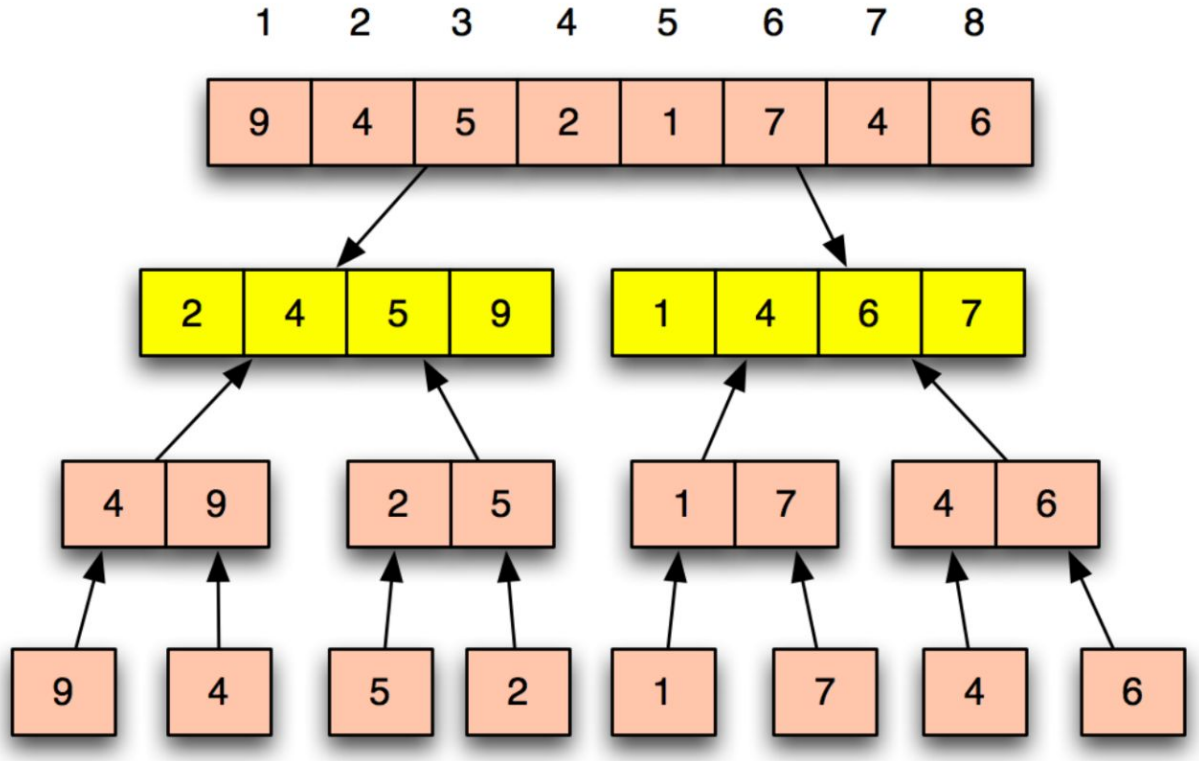
Merge Sort (Simulation)



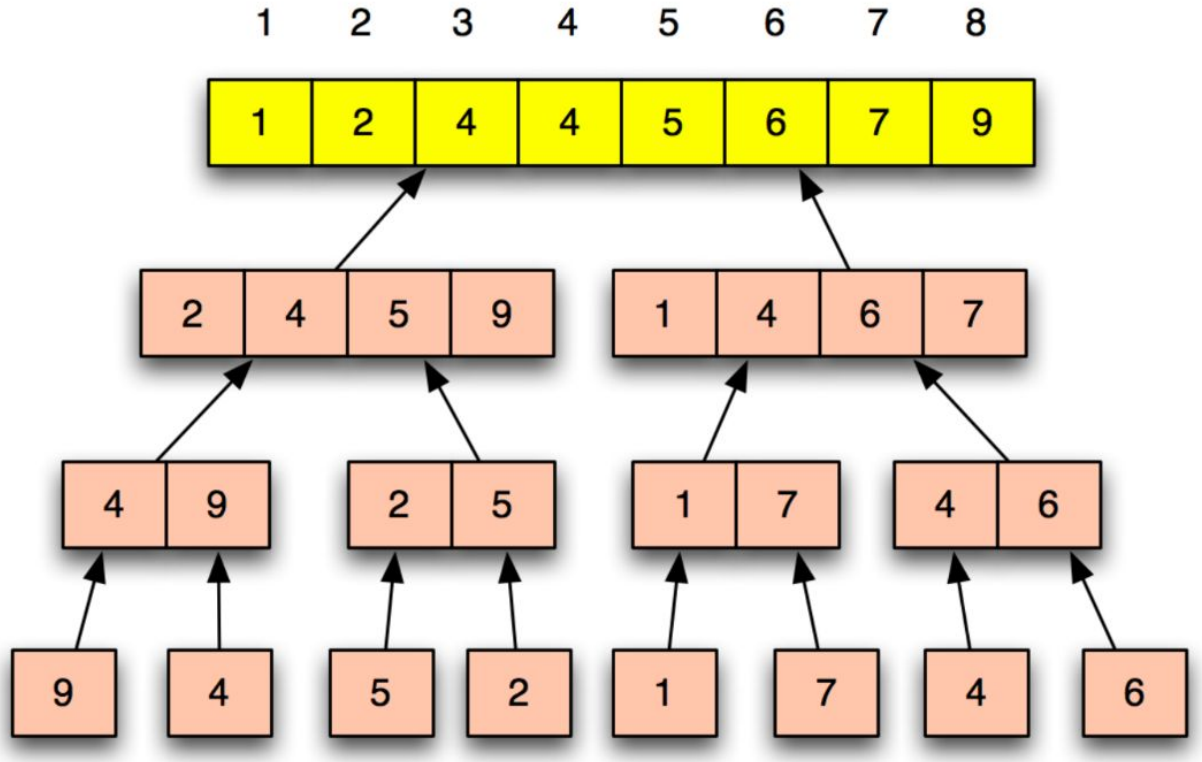
Merge Sort (Simulation)



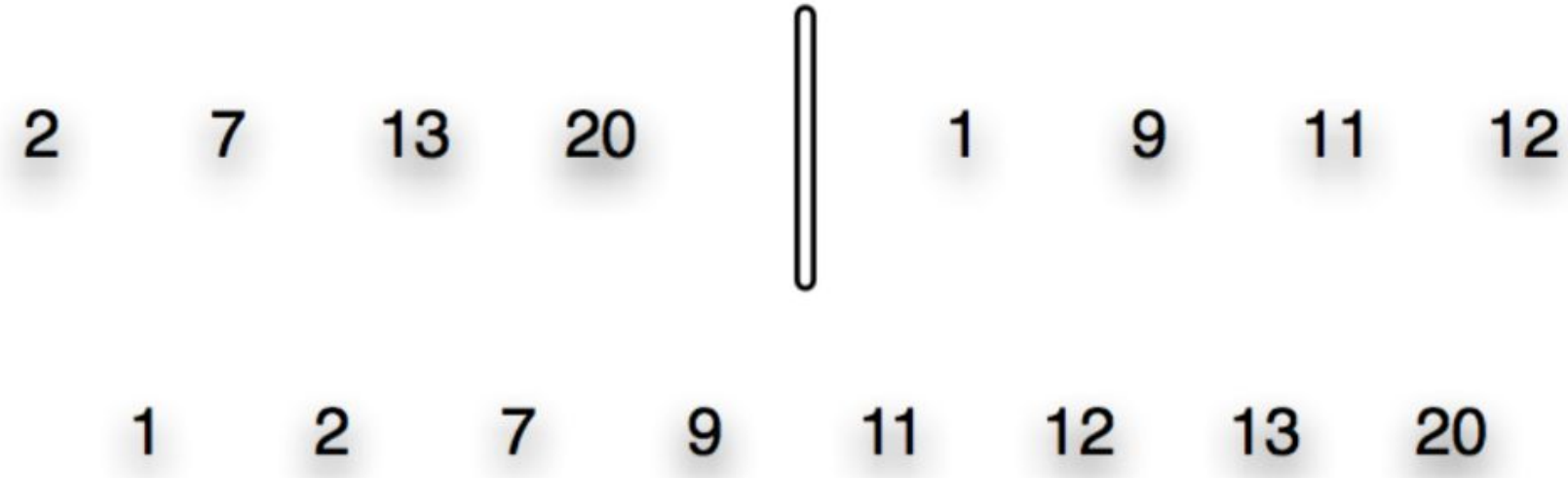
Merge Sort (Simulation)



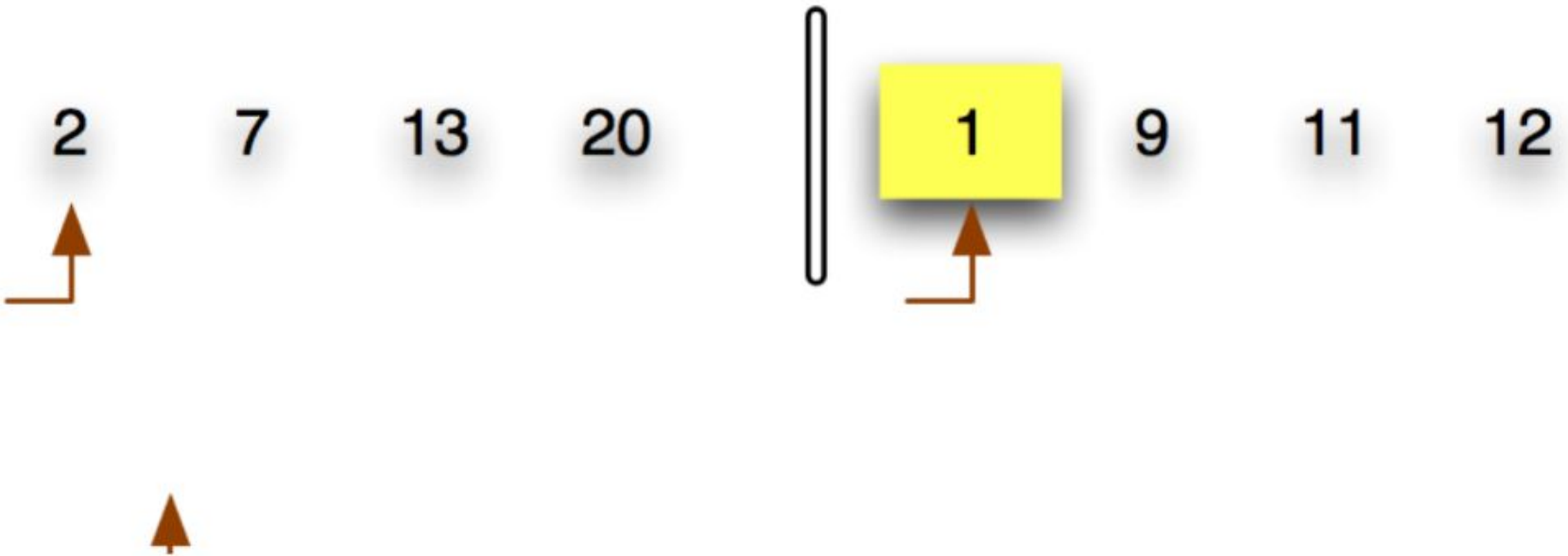
Merge Sort (Simulation)



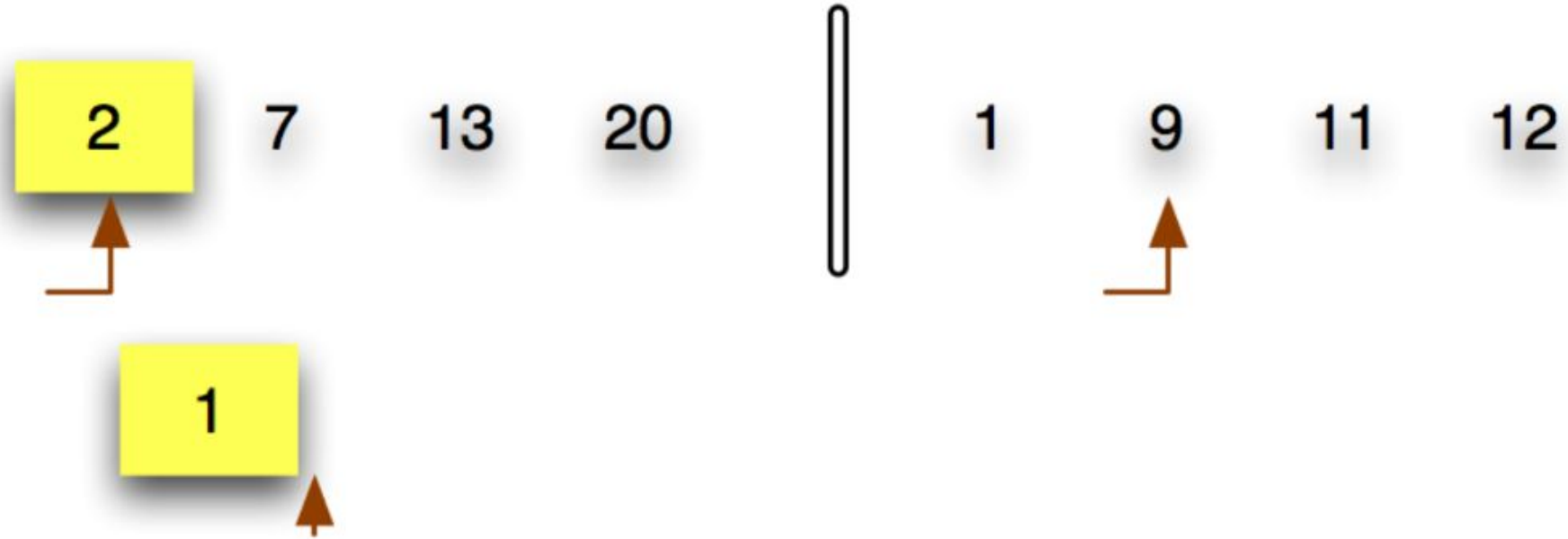
Merge (Simulation)



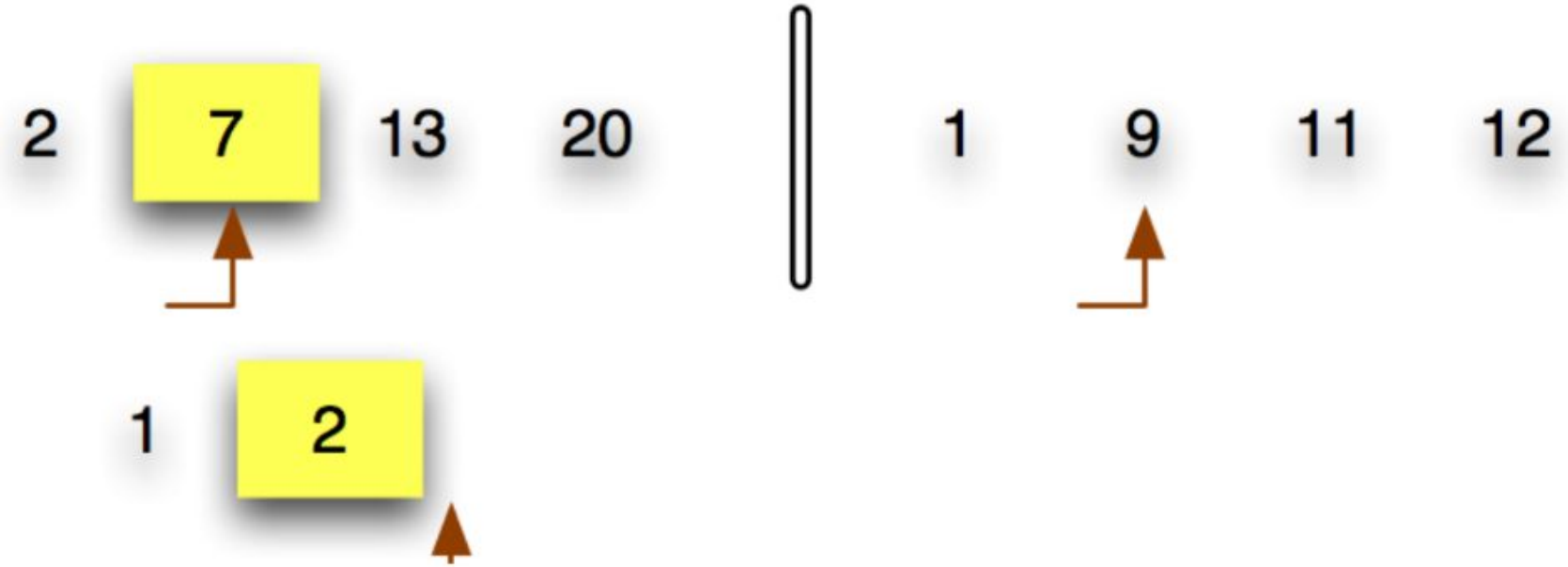
Merge (Simulation)



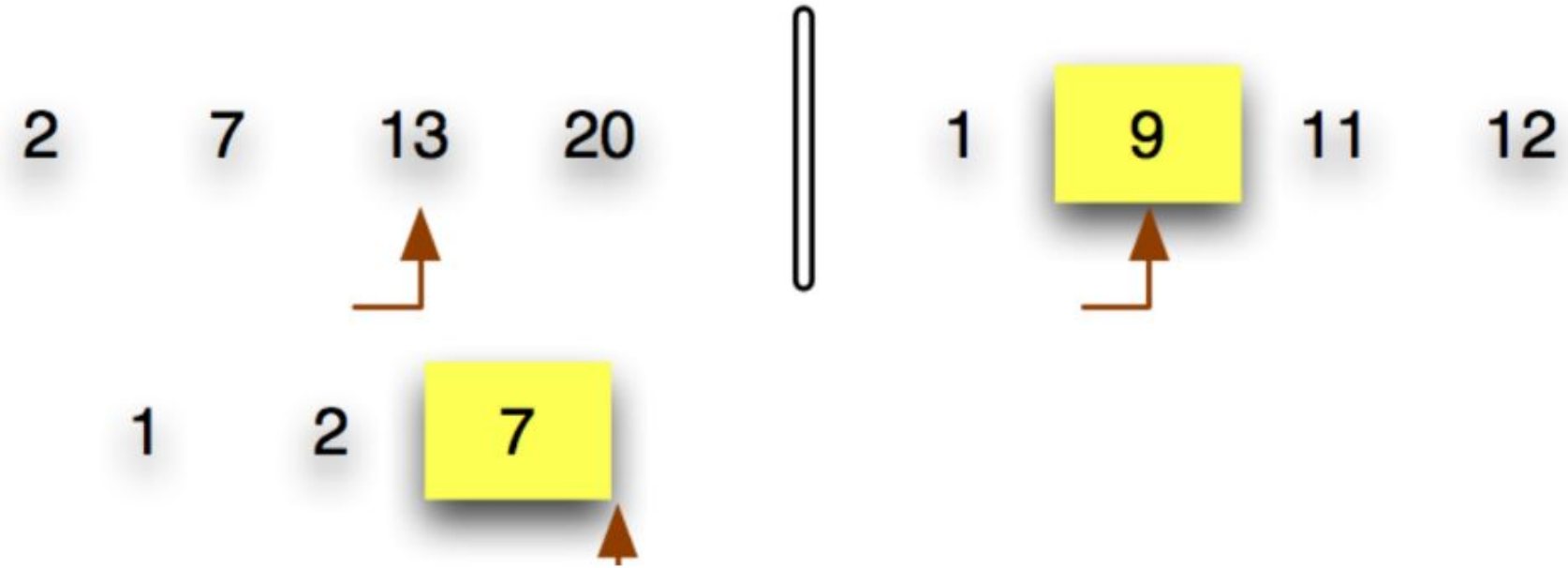
Merge (Simulation)



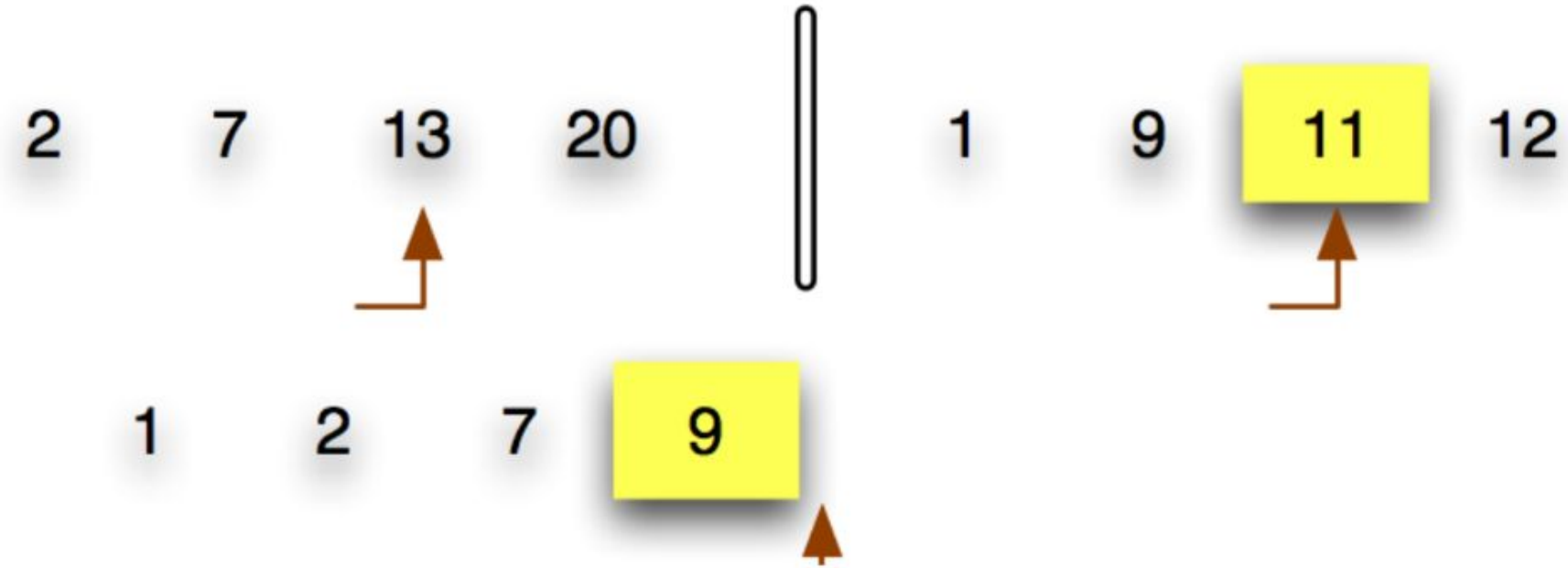
Merge (Simulation)



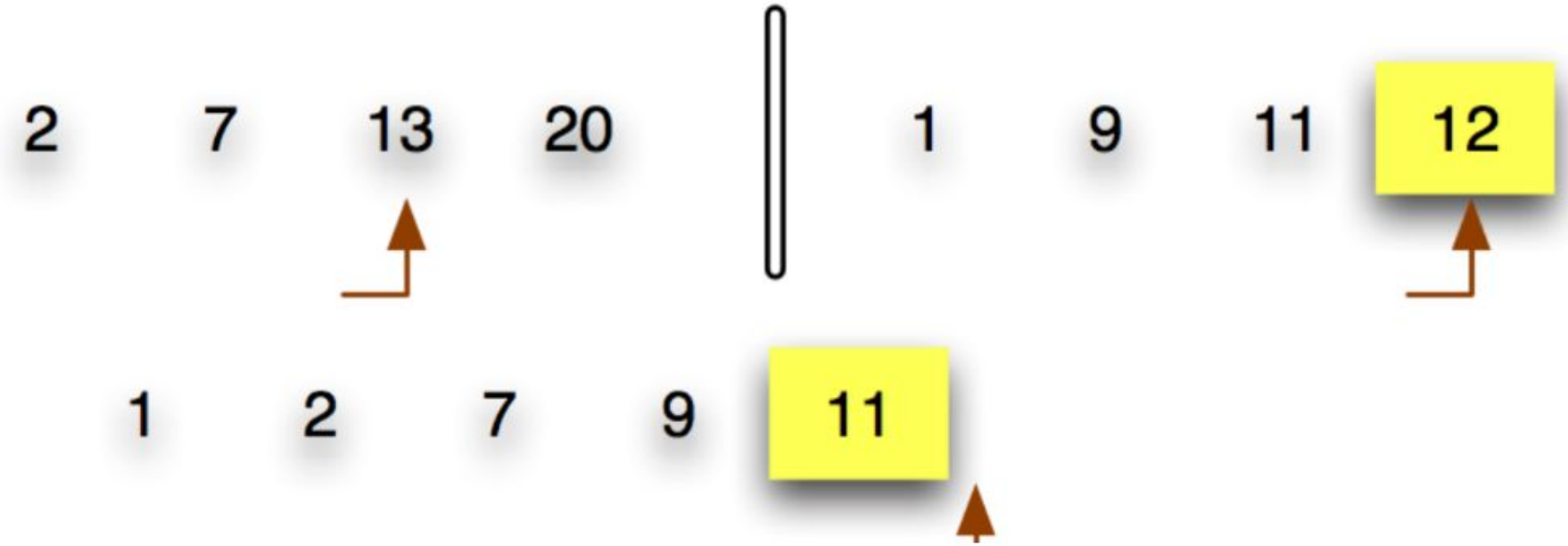
Merge (Simulation)



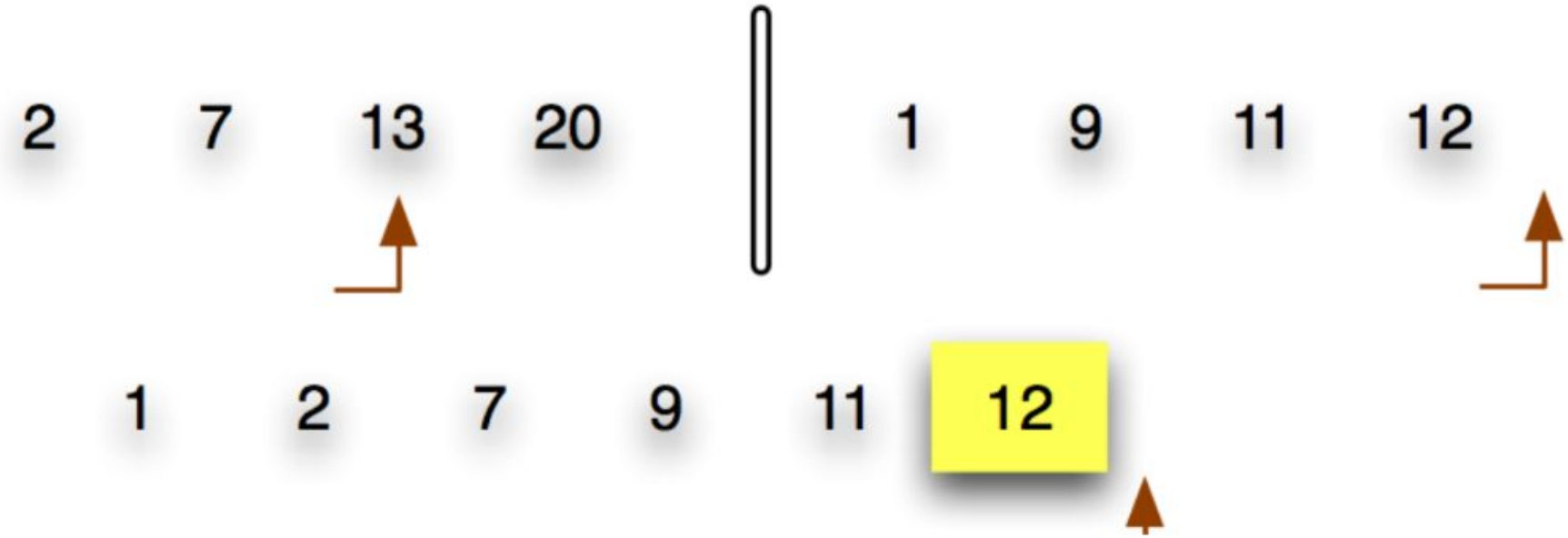
Merge (Simulation)



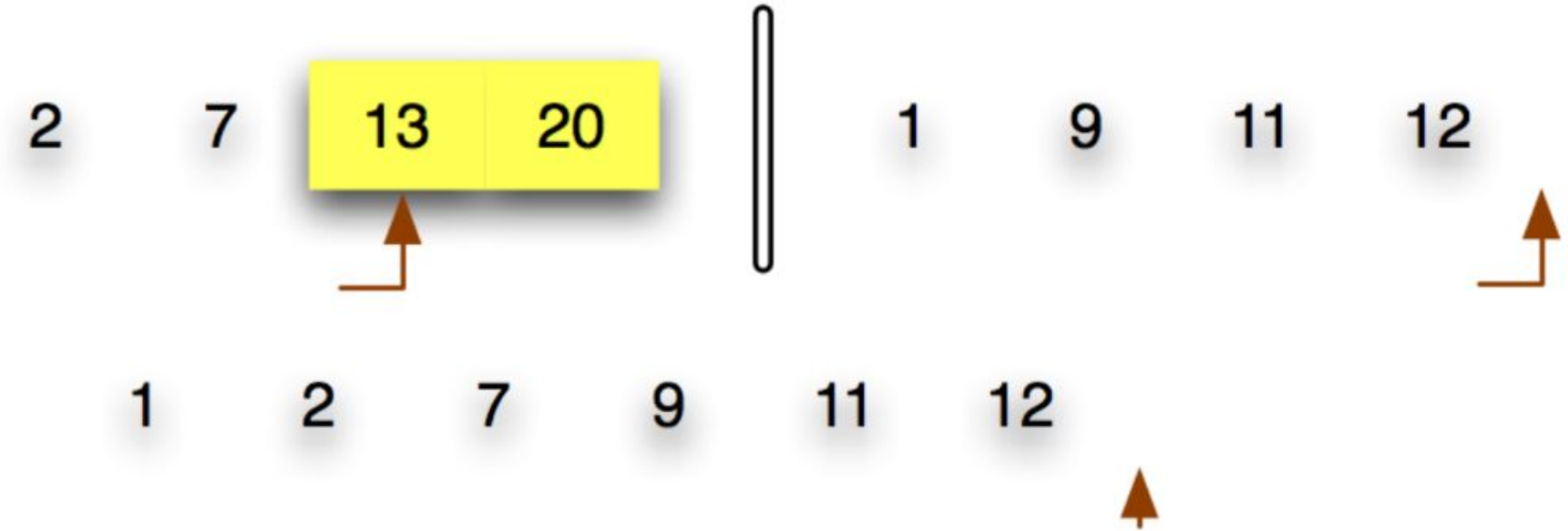
Merge (Simulation)



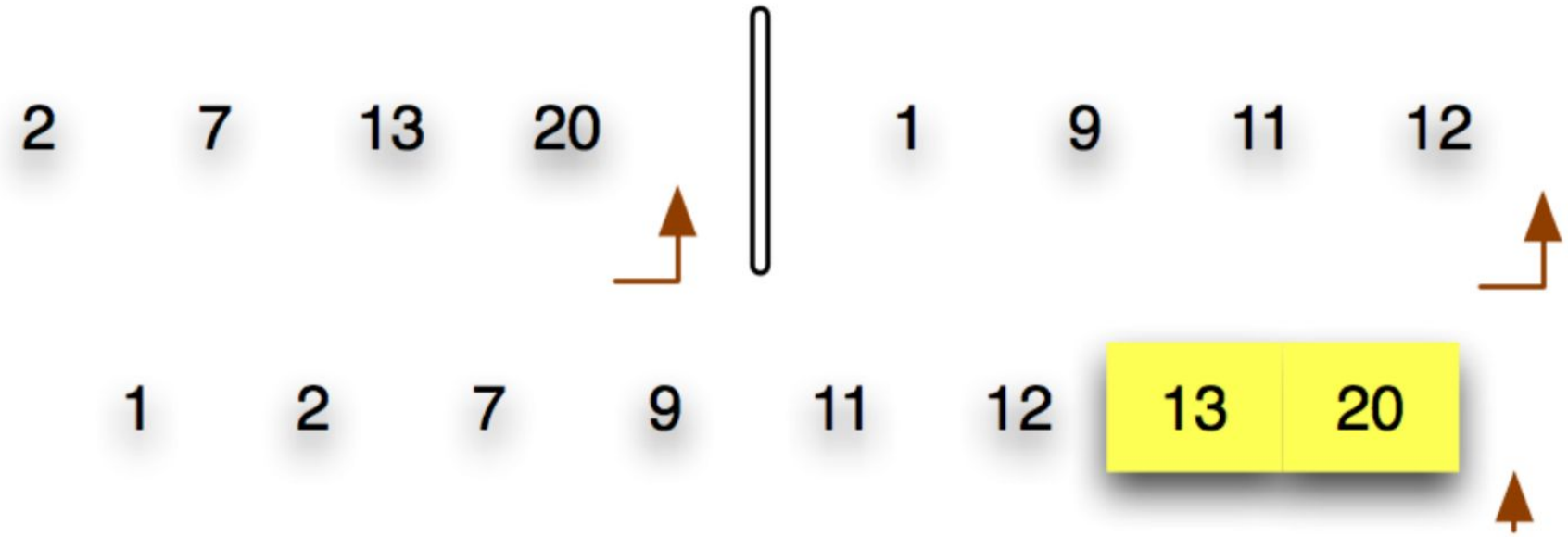
Merge (Simulation)



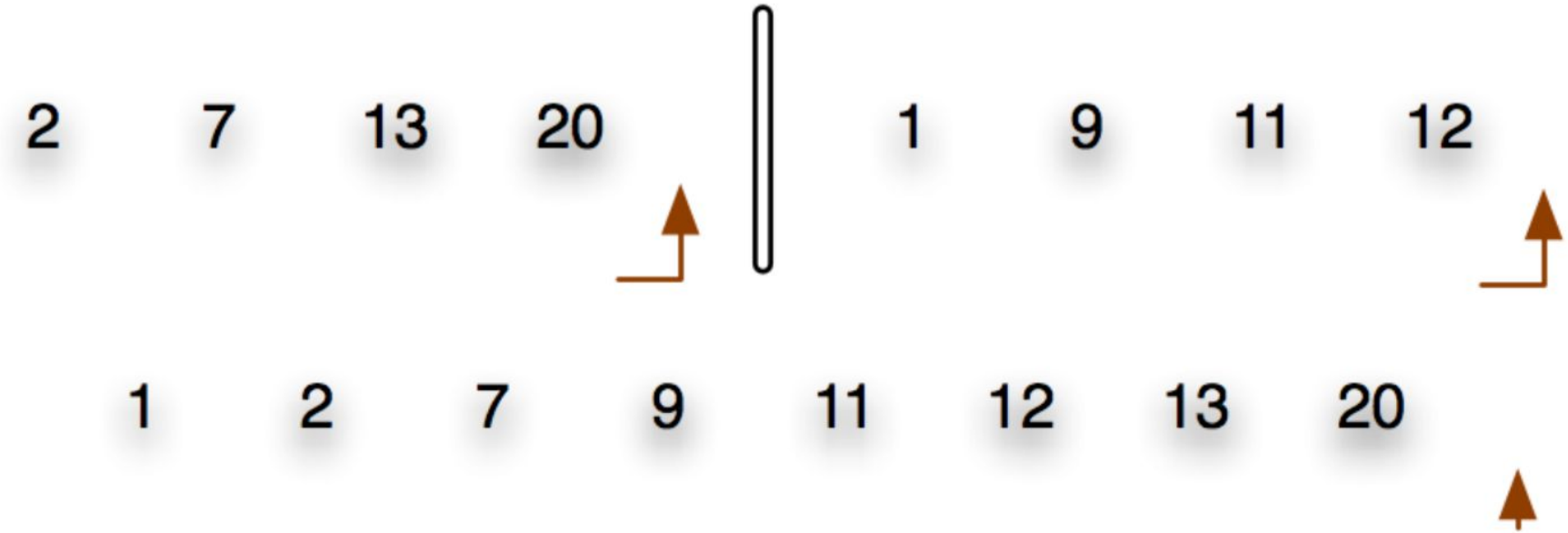
Merge (Simulation)



Merge (Simulation)



Merge (Simulation)



Merge (Algorithm)

MERGE(A, B)

INPUT: Two sorted arrays A and B
OUTPUT: Returns C as the merged array
▷ $n_1 = length[A]$, $n_2 = length[B]$, $n = n_1 + n_2$
1 Create $C[1 \,..\, n]$
2 Initialize two indices to point to A and B
3 **while** A and B are not empty
4 **do** Select the smaller of two and add to end of C
5 Advance the index that points to the smaller one
6 **if** A or B is not empty
7 **then** Copy the rest of the non-empty array to the end of C
8 **return** C

Merge Sort (Algorithm)

Out-of-Place Algorithm

```
MERGE-SORT(A) ▷ A[1 . . n]
1   if n = 1
2       then return
3       else                                ▷ recursively sort the two subarrays
4            A_1 = MERGE-SORT(A[1 . . ⌈n/2⌉])
5            A_2 = MERGE-SORT(A[⌈n/2⌉ + 1 . . n])
6            A = MERGE(A_1, A_2)            ▷ merge the sorted arrays
```

Recurrence for merge sort

$$T(n) = 2T(n/2) + \Theta(n)$$

subproblems

subproblem size

work dividing and combining

Quick Sort

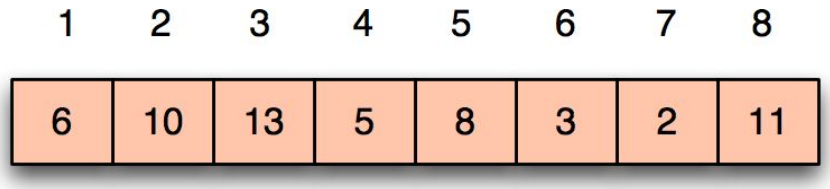
Quicksort an n-element array:

1. *Divide* Partition the array into subarrays around a *pivot* x such that the elements in lower subarray $\leq x \leq$ elements in the upper subarray.

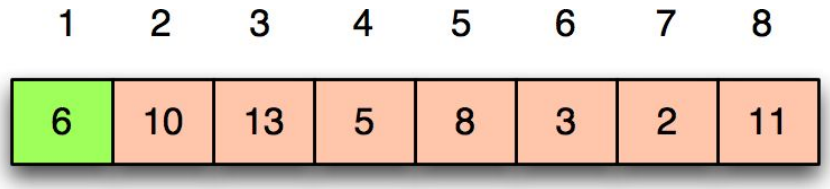


2. *Conquer* Recursively sort the two subarrays.
3. *Combine* Trivial – just concatenate the lower subarray, *pivot*, and the upper subarray.

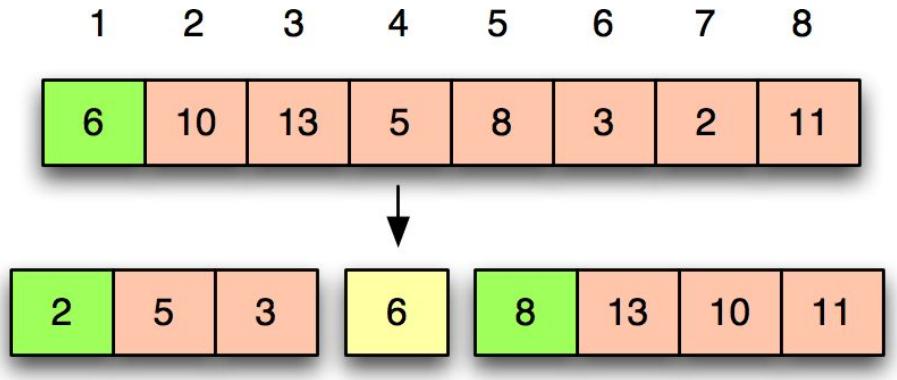
Quick Sort (Simulation)



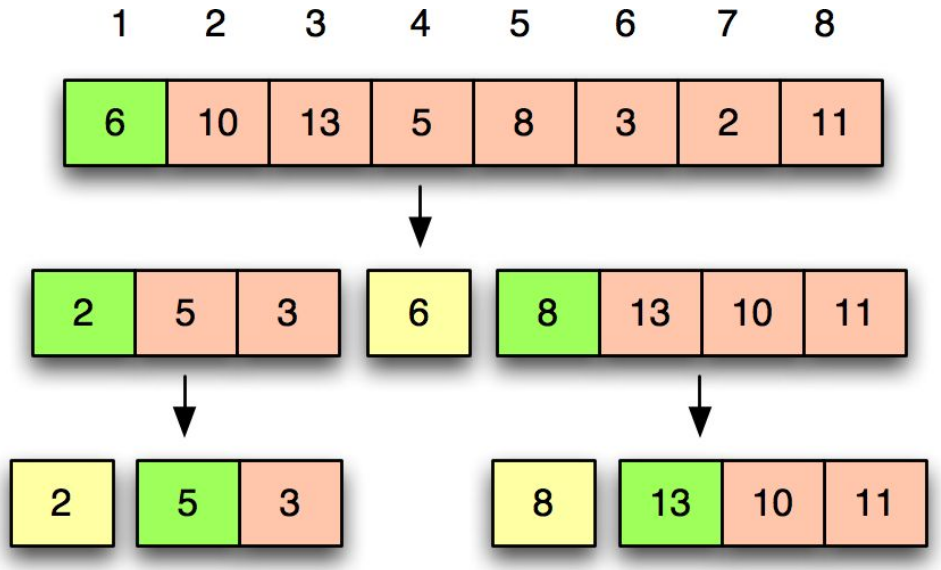
Quick Sort (Simulation)



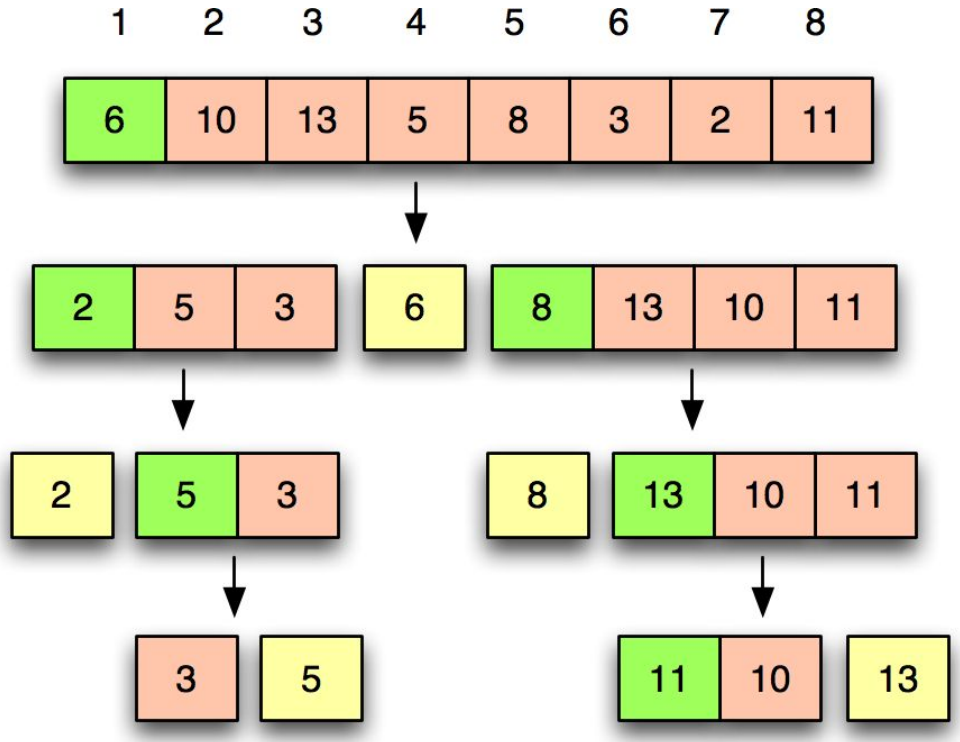
Quick Sort (Simulation)



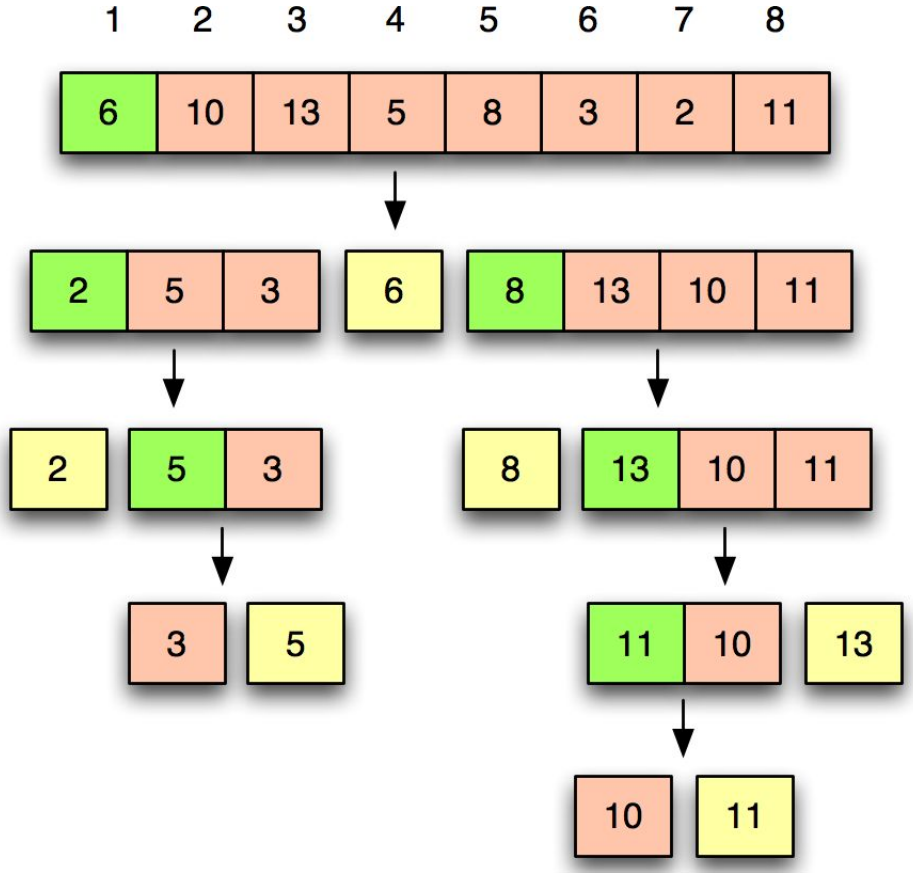
Quick Sort (Simulation)



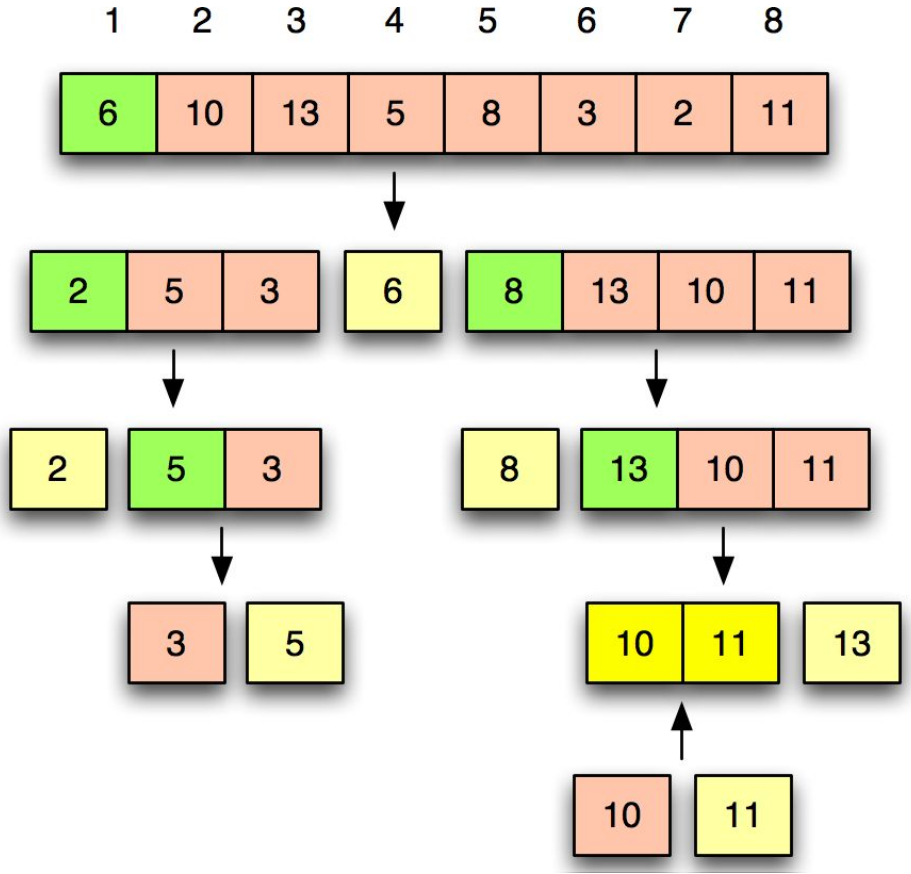
Quick Sort (Simulation)



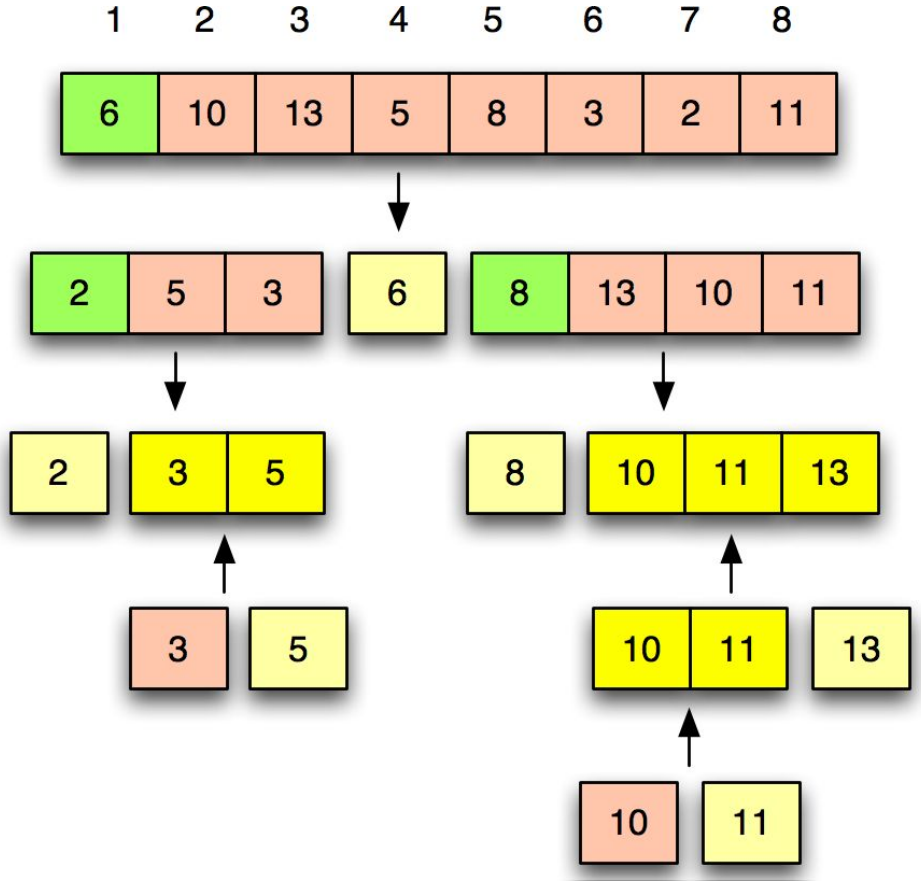
Quick Sort (Simulation)



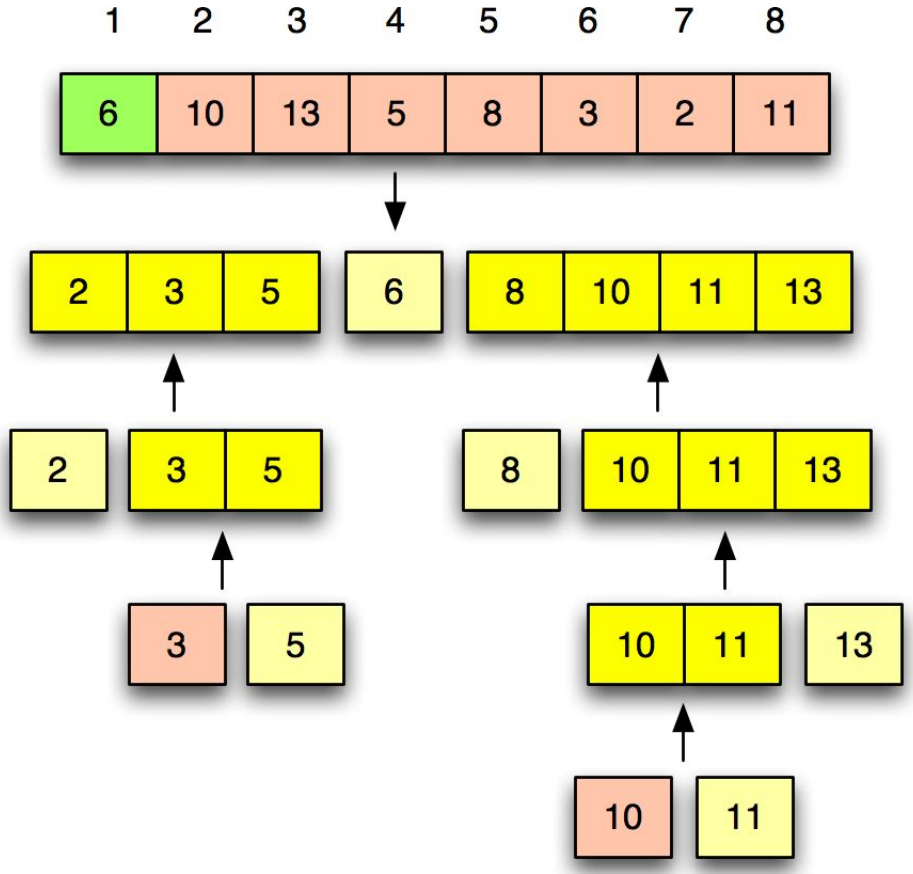
Quick Sort (Simulation)



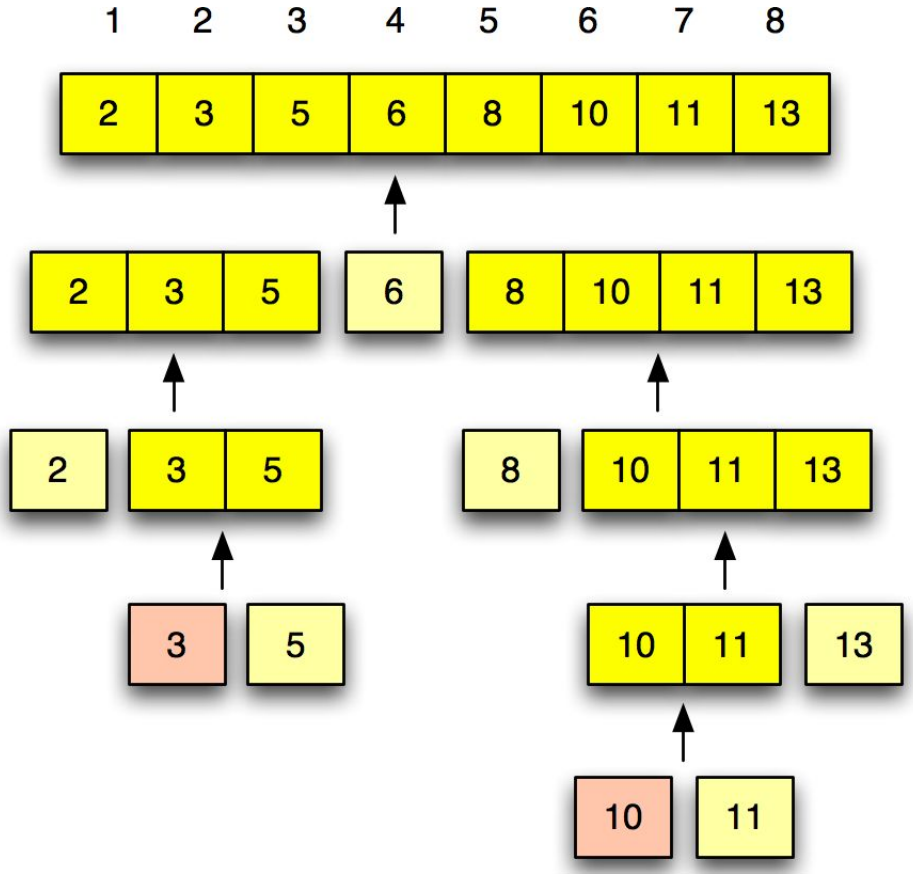
Quick Sort (Simulation)



Quick Sort (Simulation)



Quick Sort (Simulation)



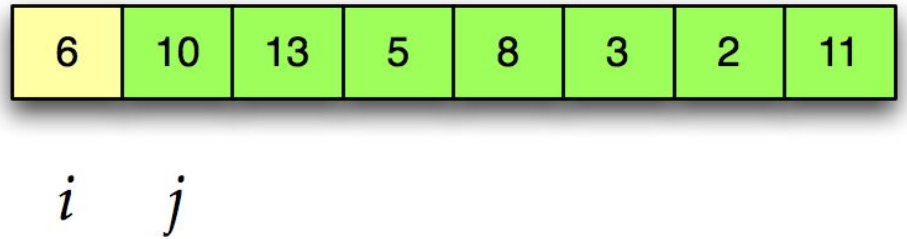
Partitioning (Algorithm)

Algorithm

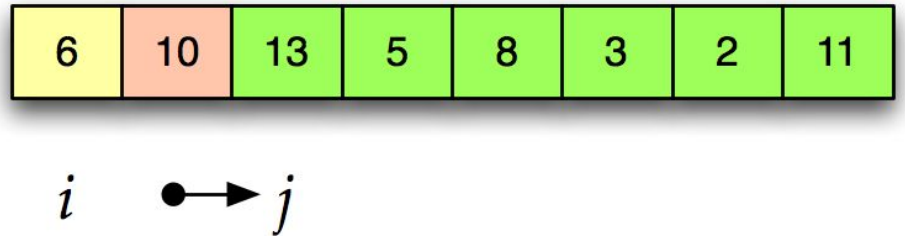
$\textsc{partition}(A, p, q) \triangleright A[p\,..\,q]$

```
1  x ← A[p]              ▷ pivot = A[p]
2  i ← p
3  for j ← p + 1 to q
4        do if A[j] ≤ x
5              then i ← i + 1
6                   exchange A[i] ↔ A[j]
7  exchange A[p] ↔ A[i]
8  return i
```

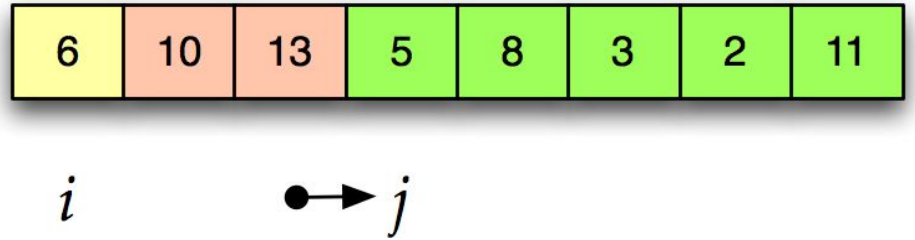
Partitioning (Simulation)



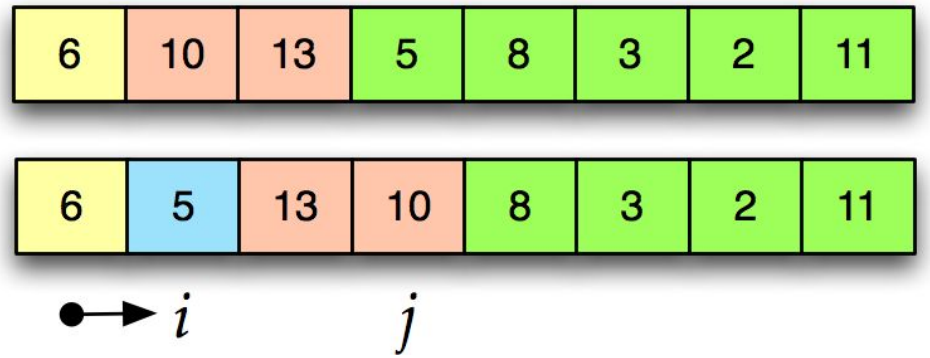
Partitioning (Simulation)



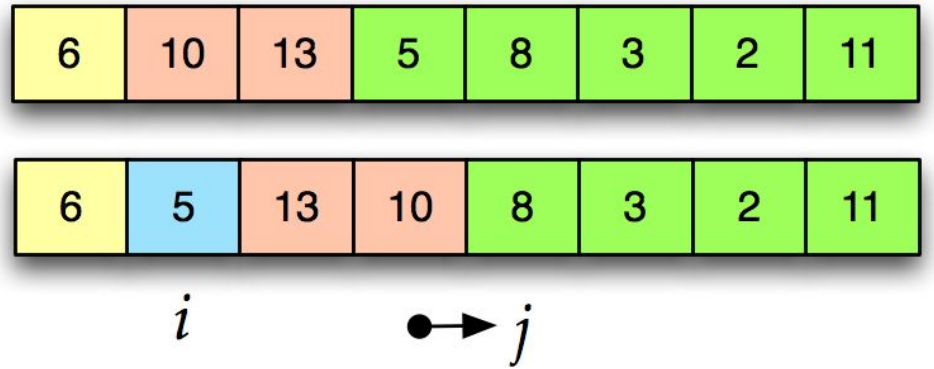
Partitioning (Simulation)



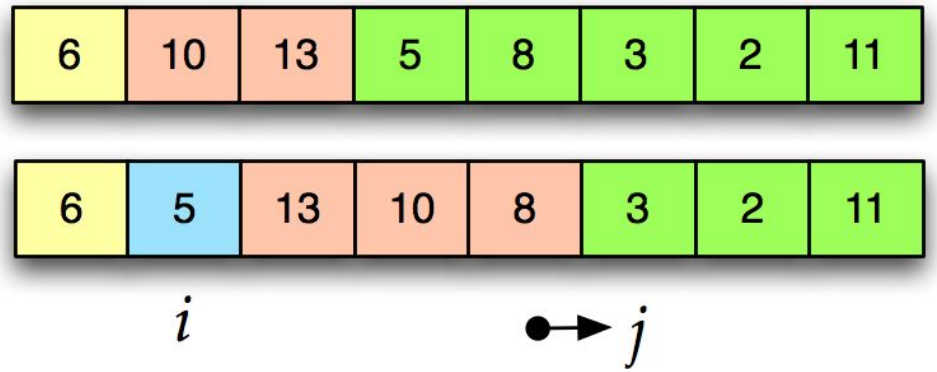
Partitioning (Simulation)



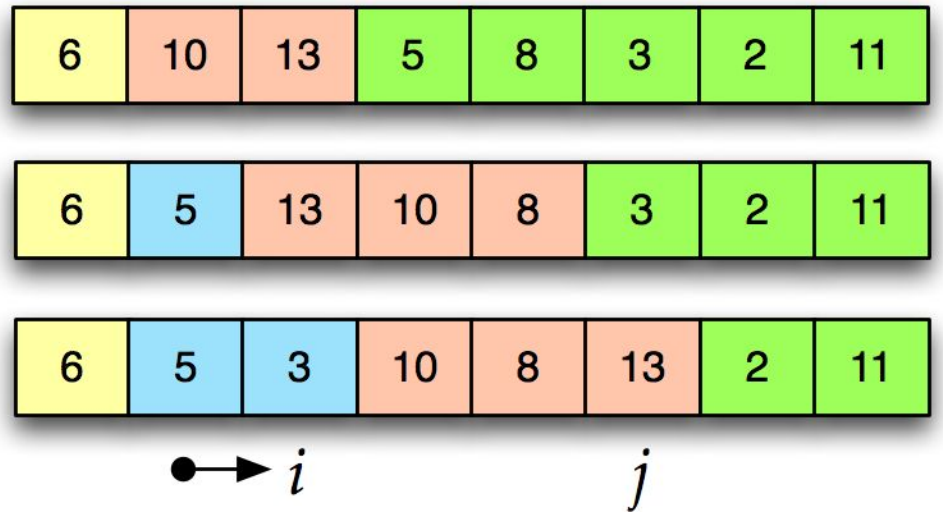
Partitioning (Simulation)



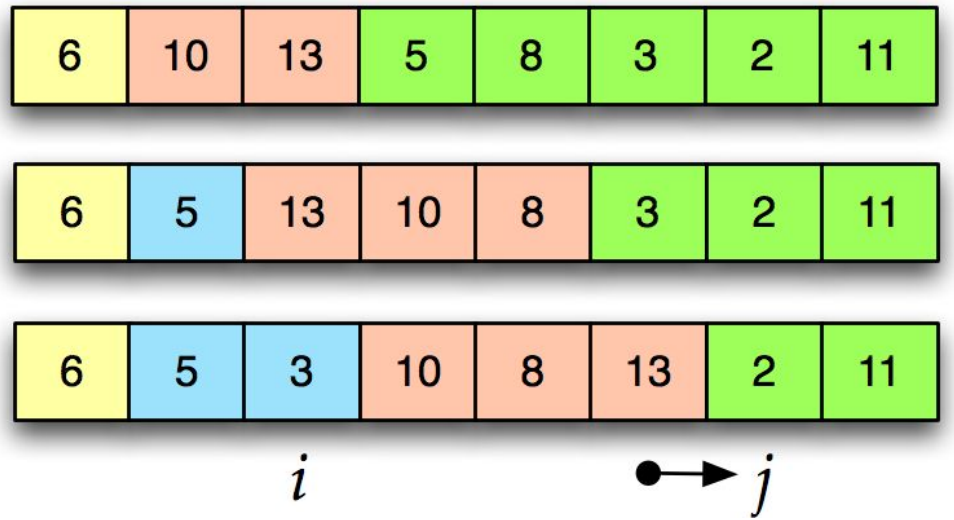
Partitioning (Simulation)



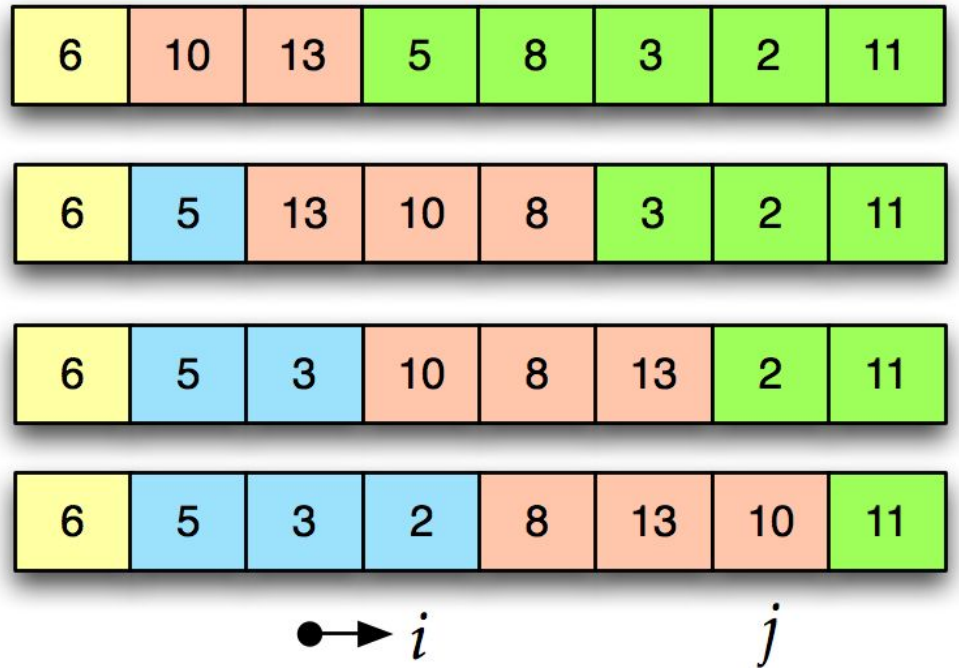
Partitioning (Simulation)



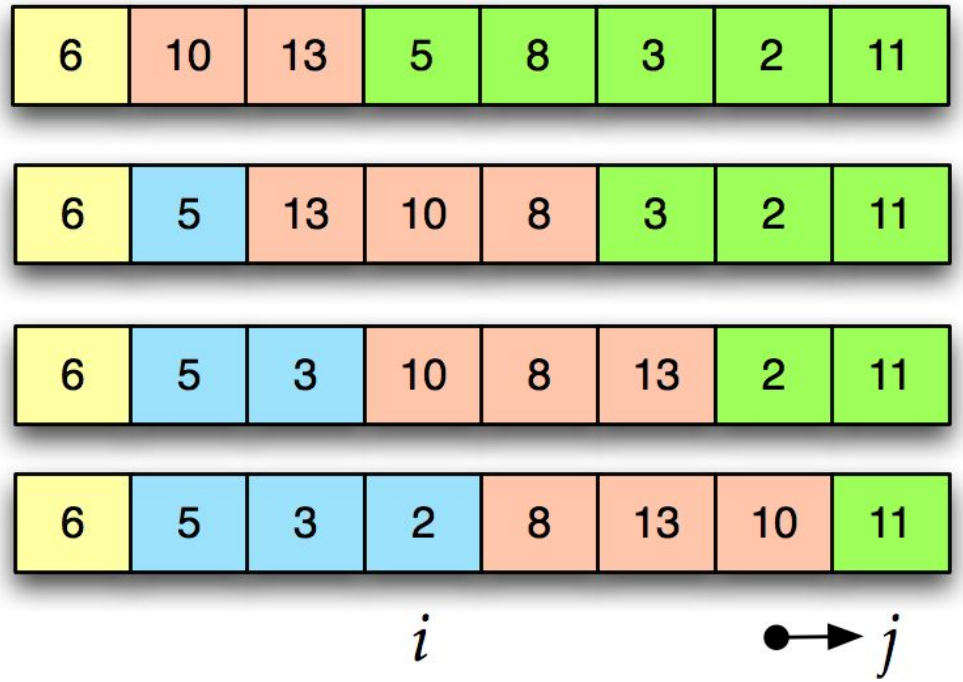
Partitioning (Simulation)



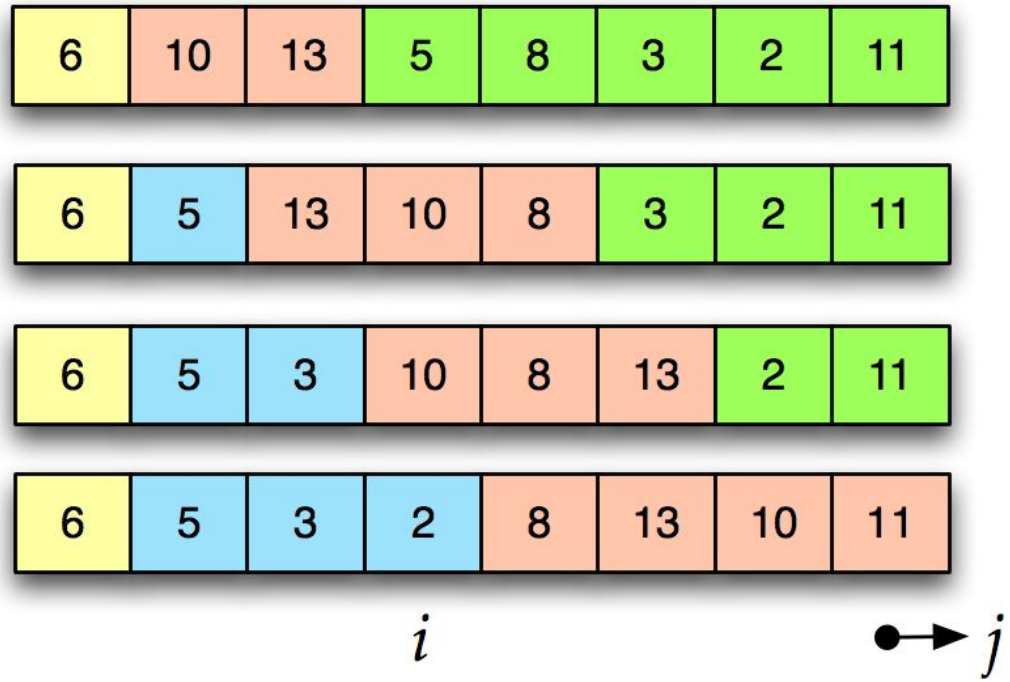
Partitioning (Simulation)



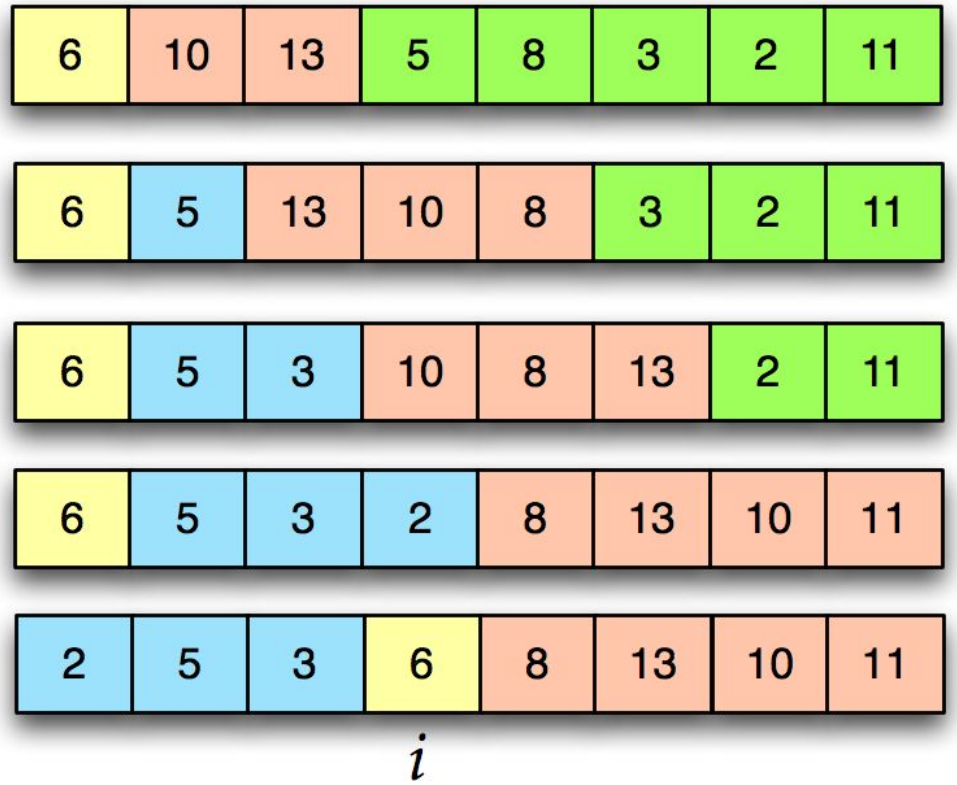
Partitioning (Simulation)



Partitioning (Simulation)



Partitioning (Simulation)



Quick Sort (Algorithm)

Algorithm

$\textsc{quicksort}(A, p, r) \triangleright A[p\,..\,r]$

1 **if** $p < r$
2 **then** $q \leftarrow \textsc{partition}(A, p, r)$
3 $\textsc{quicksort}(A, p, q-1)$
4 $\textsc{quicksort}(A, q+1, r)$

Quick Sort (Soft Analysis - Worst Case)

- Worst-case happens when pivot is always the minimum or maximum element.

Quick Sort (Soft Analysis - Best Case)

- Best-case happens when pivot is the median element, creating equal size partitions.

Randomized version of quicksort

```
RANDOMIZED-PARTITION(A, p, r)
1  i = RANDOM(p, r)
2  exchange A[r] with A[i]
3  return PARTITION(A, p, r)
```

The new quicksort calls RANDOMIZED-PARTITION in place of PARTITION:

```
RANDOMIZED-QUICKSORT(A, p, r)
1  if p < r
2       q = RANDOMIZED-PARTITION(A, p, r)
3       RANDOMIZED-QUICKSORT(A, p, q − 1)
4       RANDOMIZED-QUICKSORT(A, q + 1, r)
```

Counting Sort

- It is a linear time sorting!
- Complexity : O(n) | $O(n + k)$ to be precise.
  - O(n) when k << n, where k is the range of the values in input array
- It is not a comparison sort
- It is a stable sorting algorithm
- Only suitable for direct use in situations where the variation in keys is not significantly greater than the number of items.

Idea of Counting sort

- Suppose you know prior hand that all the values in the array are within 1 and k.
- Take k boxes, 1 for each different value.
- Put the value i in box i.
- Each box contains elements equal to the frequency of the corresponding element
- For sorting in increasing order, take out values from the smallest box to the largest box and place them in a new array, which will provide the sorted sequence

Implementation example

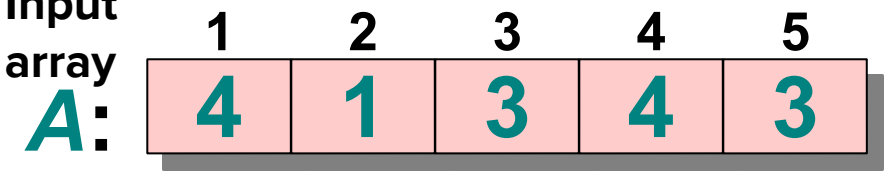
- Array A : Main array
- Array C : Counting box array
- Array C’ : Keep cumulative sum of C
- Array B : Build the sorted array

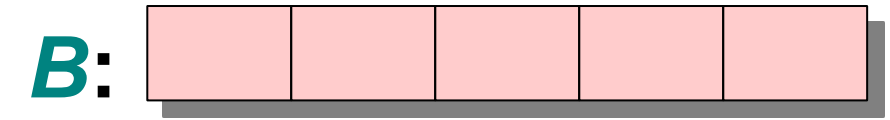
Pseudocode of Counting Sort

```
void counting_sort(int A[], int k, int n)
{
   C[1…k] = 0; // initialize all elements in C array to 0
   for (j = 1; j <= n; j++)        // keep A[j] in C[A[j]]
      C[A[j]] ++;
   for (i = 1; i <= k; i++)        // cumulative sum
      C[i] = C[i] + C[i-1];        //instead of taking a separate array C', update the array C with cumulative sum
   for (j = n; j >= 1; j--)        // put the sorted array in B
   {
      B[C[A[j]]] = A[j];
      C[A[j]] --;
   }
}
```

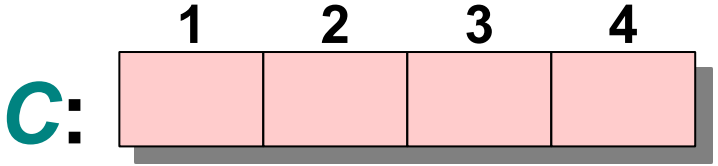
Counting-sort example

Input array



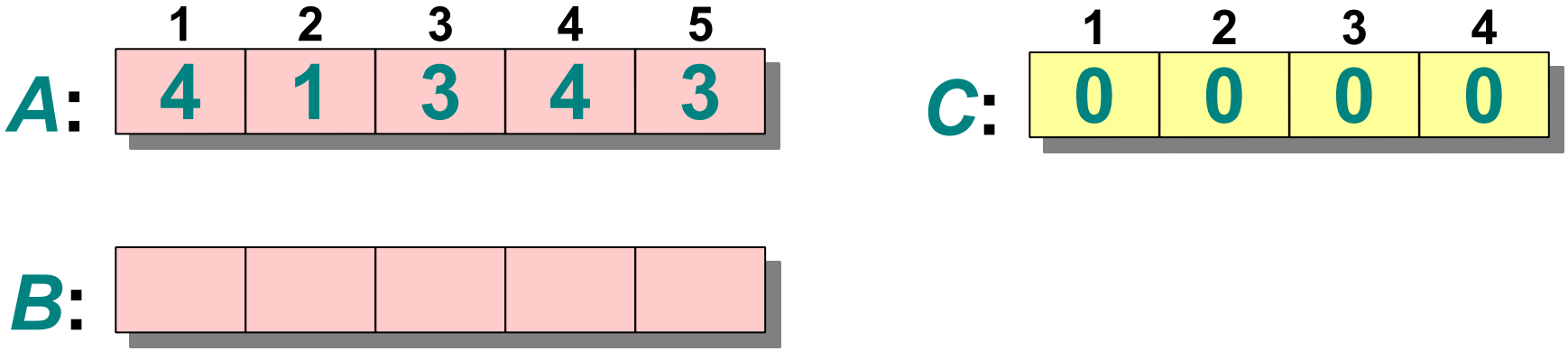


output array



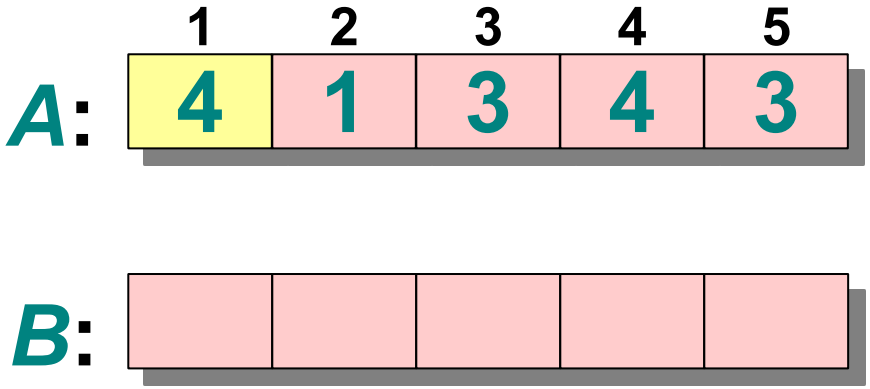
Count Array
&
cumulative Count Array

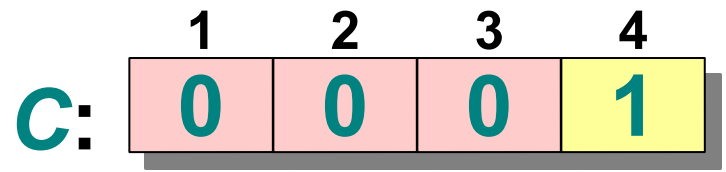
Loop 1



C[1…k] = 0; // initialize all elements in C array to 0

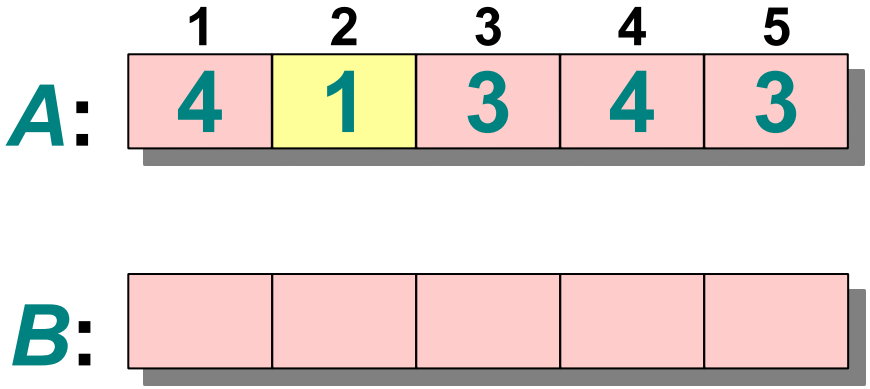
Loop 2

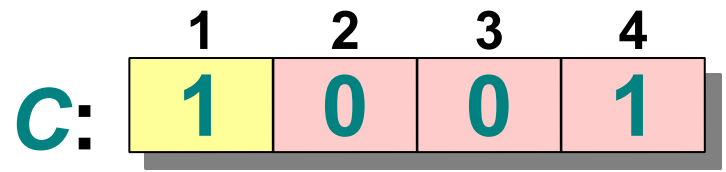





```
for (j = 1; j <= n; j++)
        C[A[j]] ++;
```

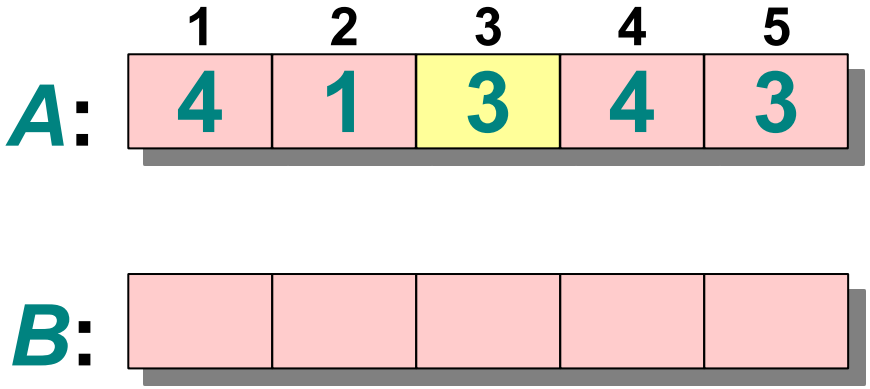
Loop 2

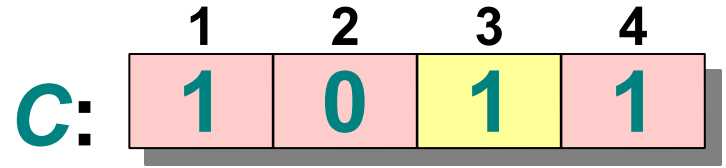





```
for (j = 1; j <= n; j++)
        C[A[j]] ++;
```

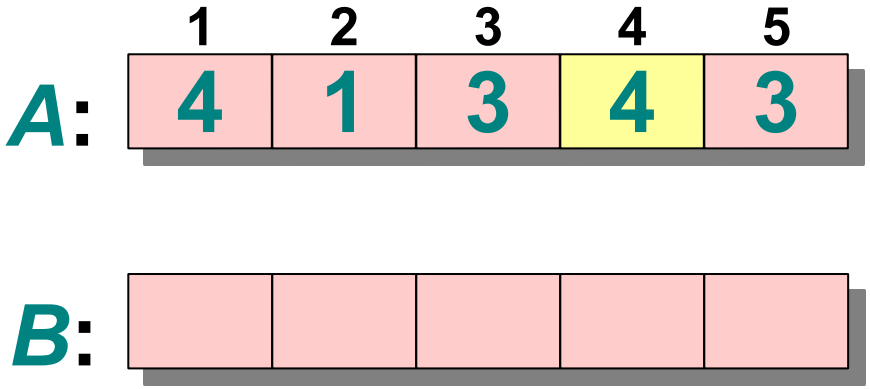
Loop 2

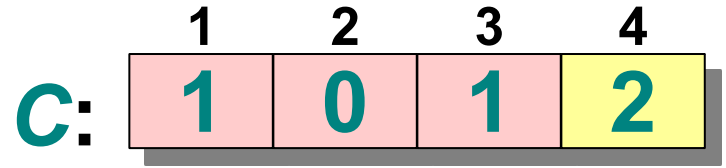





```
for (j = 1; j <= n; j++)
        C[A[j]] ++;
```

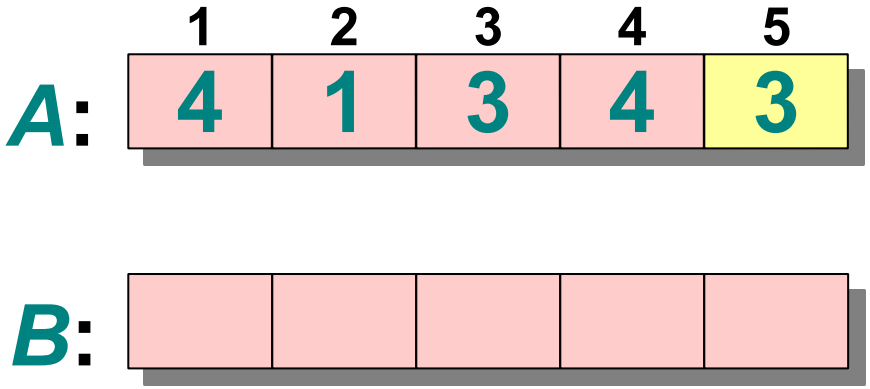
Loop 2

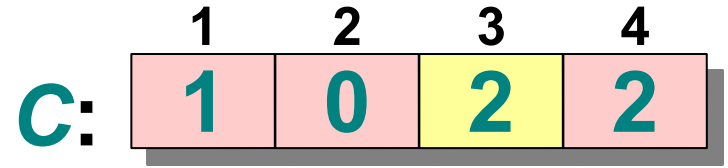





```
for (j = 1; j <= n; j++)
        C[A[j]] ++;
```

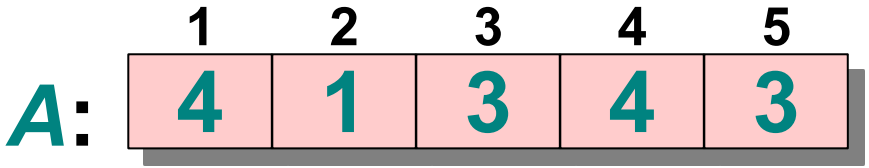
Loop 2

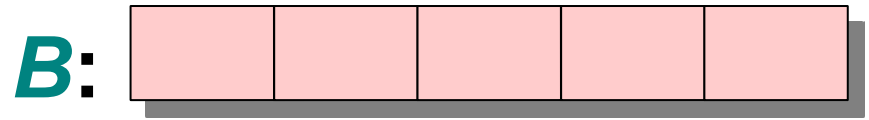


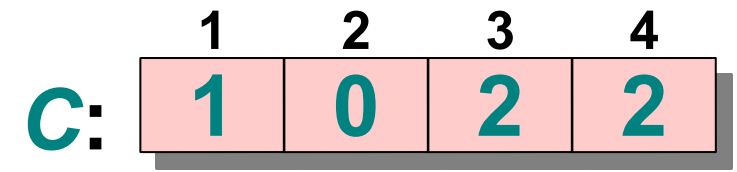



```
for (j = 1; j <= n; j++)
        C[A[j]] ++;
```

Loop 3





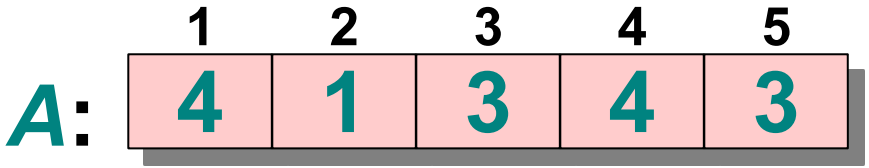


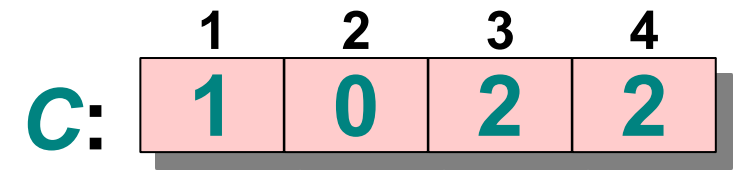


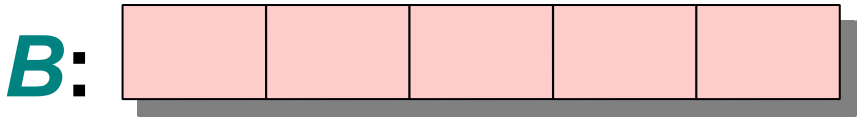

```
for (i = 1; i <= k; i++)
        C[i] = C[i] + C[i-1];
```

C' represents the state of C after C[i] = C[i] + C[i-1] operation

Loop 3





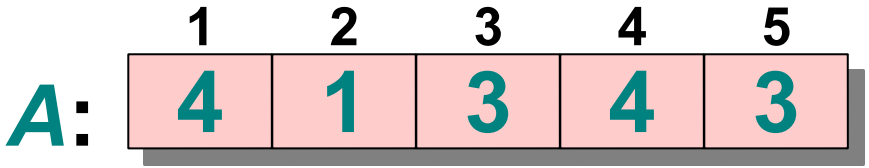


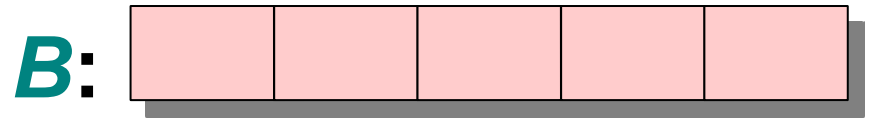


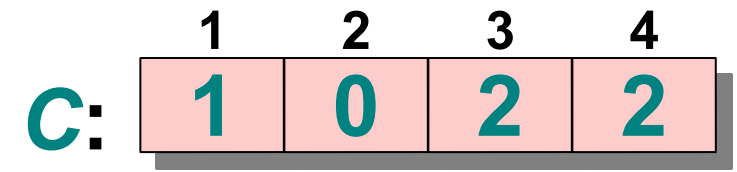

```
for (i = 1; i <= k; i++)
        C[i] = C[i] + C[i-1];
```

C' represents the state of C after C[i] = C[i] + C[i-1] operation

Loop 3





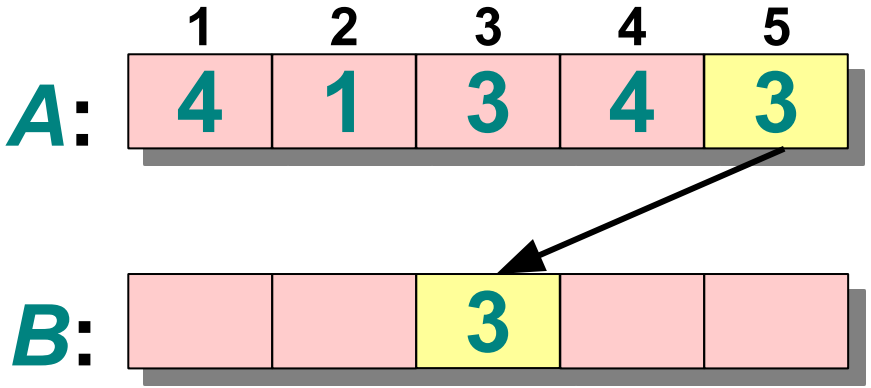


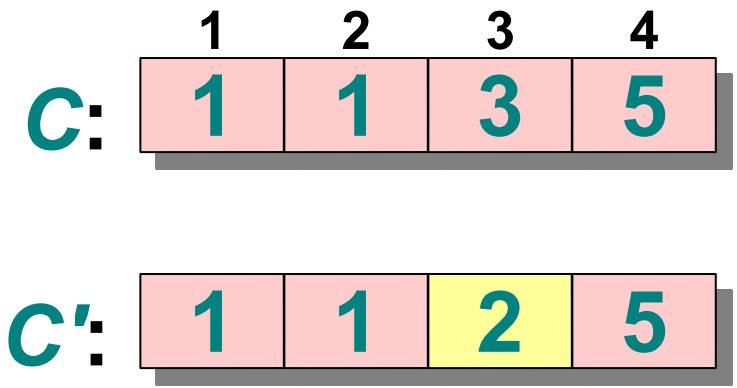



```
for (i = 1; i <= k; i++)
        C[i] = C[i] + C[i-1];
```

C' represents the state of C after C[i] = C[i] + C[i-1] operation

Loop 4

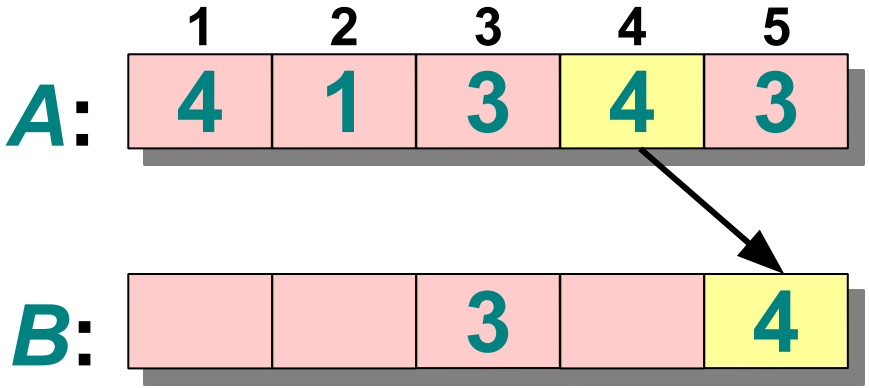


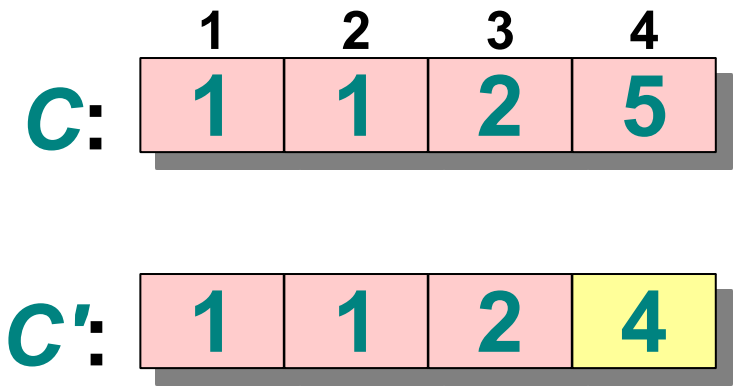



```
for (j = n; j >= 1; j--)
{
    B[C[A[j]]] = A[j];
    C[A[j]] --;
}
```

C' represents the state of C after C[A[j]]-- operation

Loop 4

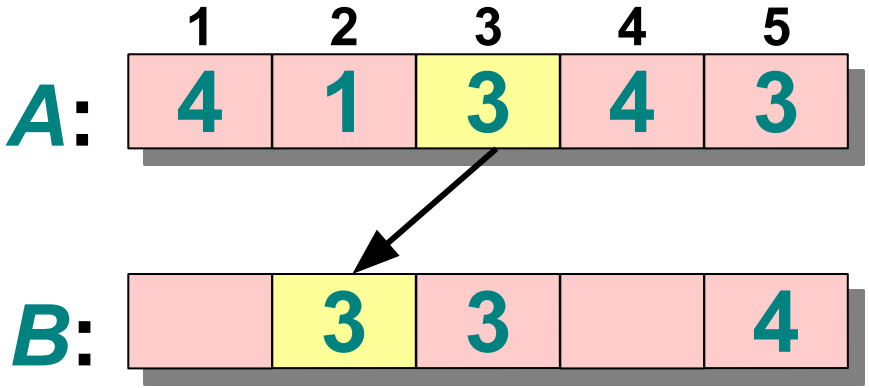


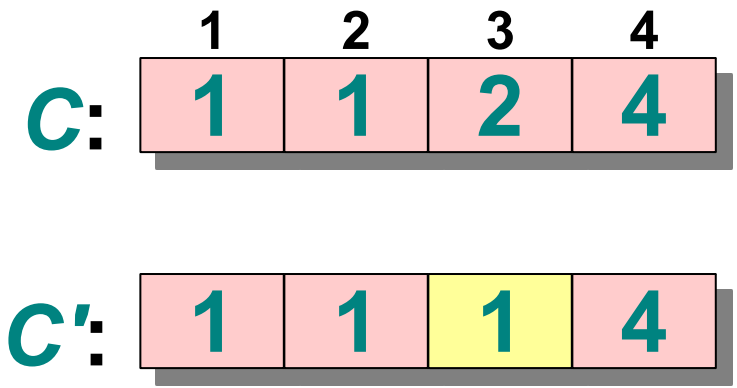



```
for (j = n; j >= 1; j--)
{
    B[C[A[j]]] = A[j];
    C[A[j]] --;
}
```

C' represents the state of C after C[A[j]]-- operation

Loop 4

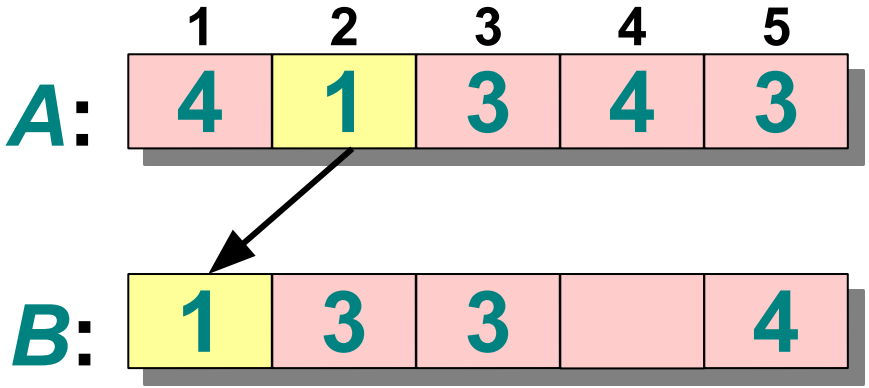


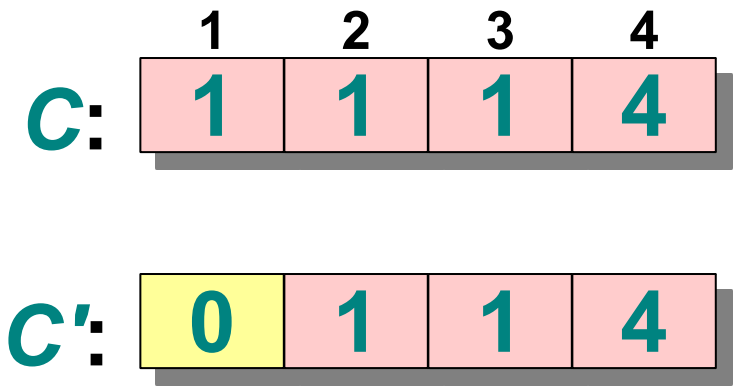



```
for (j = n; j >= 1; j--)
{
    B[C[A[j]]] = A[j];
    C[A[j]] --;
}
```

C' represents the state of C after C[A[j]]-- operation

Loop 4

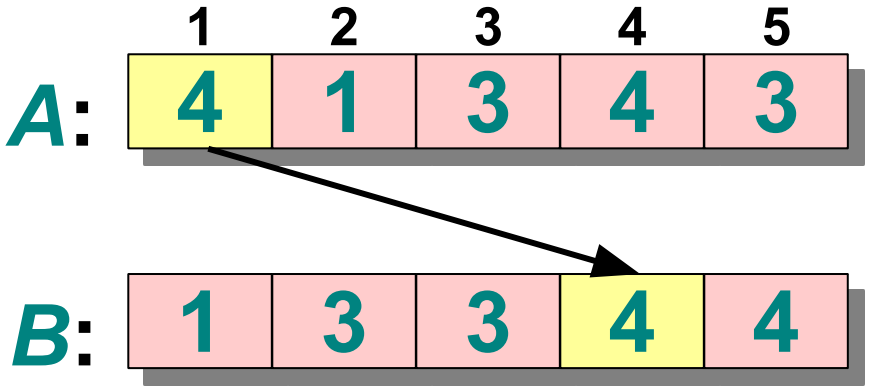


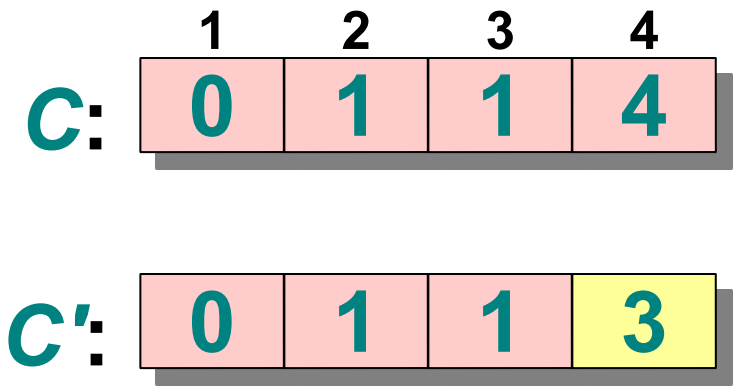



```
for (j = n; j >= 1; j--)
{
    B[C[A[j]]] = A[j];
    C[A[j]] --;
}
```

C' represents the state of C after C[A[j]]-- operation

Loop 4






```
for (j = n; j >= 1; j--)
{
    B[C[A[j]]] = A[j];
    C[A[j]] --;
}
```

C' represents the state of C after C[A[j]]-- operation

Analysis of Counting Sort

$\Theta(k)$

```
C[1...k] = 0;
```

$\Theta(n)$

```
for (j = 1; j <= n; j++)
    C[A[j]] ++;
```

$\Theta(k)$

```
for (i = 1; i <= k; i++)
    C[i] = C[i] + C[i-1];
```

$\Theta(n)$

```
for (j = n; j >= 1; j--)
{
    B[C[A[j]]] = A[j];
    C[A[j]] --;
}
```

$\Theta(n + k)$

Counting Sort Extended

- What if the numbers are ranged from 100,000,000 to 100,000,100?
- What if the numbers are ranged from -1000 to 1000?
- All we need is the range between highest and lowest number.

Properties of Counting Sort

- Time Complexity:
  - Best Case: O(n+k) when array is sorted already.
  - Average Case: O(n+k) when array is in random order.
  - Worst Case: O(n+k) when array is in reverse order.
- Space Complexity:
  - Best/Average/Worst case: O(k).
- Stable: Yes.
- Inplace: No.
- Suitable for arrays with small range of data i.e. where the difference between the largest and the smallest element of the array is not very big.